

**SECOND ASSIGNMENT REPORT (FINAL PROJECT)
REVISION**

Forming A PCG Signal Reconstruction Using Linear Prediction



Name : Jeremia Christ Immanuel Manalu
NRP : 5023231017
Course : Biomodelling (A)
Class : A
Lecturer : Eko Agus Suprayitno, S.Si., M.T.
Department : Biomedical Engineering

**FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS
TECHNOLOGY
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
2025**

CHAPTER I. FUNDAMENTAL THEORY

1.1. Phonocardiogram (PCG)

A Phonocardiogram (PCG) is a graphical representation of the sounds produced by the heart during a cardiac cycle. It is a non-invasive diagnostic tool that records the acoustic phenomena associated with the heart's mechanical activity. The primary components of a PCG signal are the first heart sound (S1), which corresponds to the "lub" sound, and the second heart sound (S2), which corresponds to the "dub" sound.

- S1 (Lub) that occurs at the beginning of systole and is associated with the closure of the mitral and tricuspid valves.
- S2 (Dub) that occurs at the end of systole and is associated with the closure of the aortic and pulmonary valves.

As a time-series signal, the PCG contains both periodic and quasi-random components. The repetitive nature of the S1-S2 cycle makes it a suitable candidate for analysis using predictive models, which can help in identifying patterns, compressing data, or detecting abnormalities.

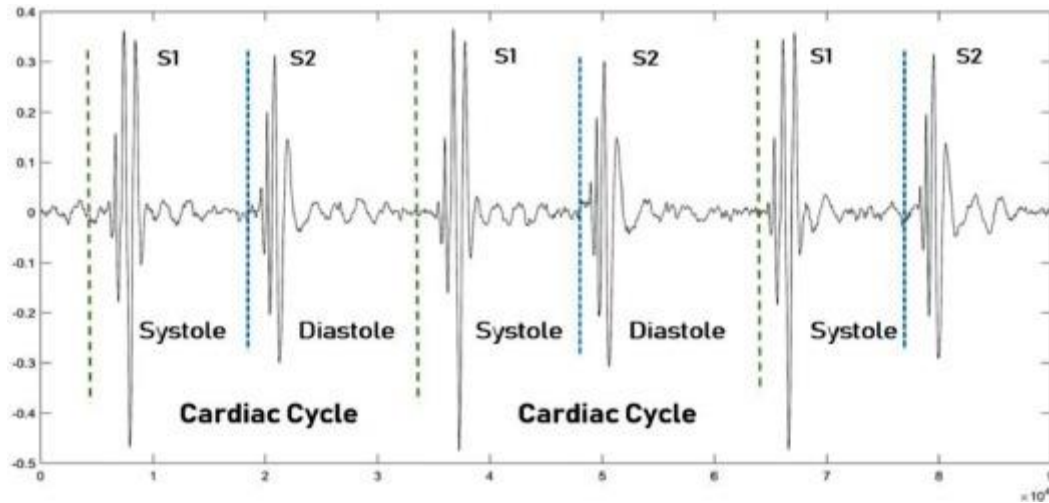


Figure 1.1. An example of PCG Signal

1.2. Autocorrelation

Autocorrelation is a mathematical tool used to measure the self-similarity of a signal over different time delays. It involves comparing a signal with a time-shifted version of itself. For a discrete signal $x(m)$, the autocorrelation function $r_{xx}(l)$ for a given time lag l is calculated by summing the product of the signal and its shifted version.

$$r_{xx}(l) = \sum_{m=0}^{M-1} x[m] * x[m - l]$$

Where:

- $r_{xx}(l)$ is the autocorrelation value at lag l .
- $x[m]$ is the value of the signal at sample index n .
- M is the total number of samples (referred to as Sample in the code).
- l is the time lag.

1.3. Linear Prediction

Linear Prediction is a signal processing technique used to forecast future values of a signal based on a linear combination of its past values. The core idea is that a significant portion

of a signal can be predicted from its previous samples, especially if the signal has inherent correlations. The predicted value of the signal, $\hat{x}(m)$, at time m is calculated as a weighted sum of P previous samples:

$$\hat{x}(m) = \sum_{k=1}^P a[k] * x[m - k]$$

Here:

- $x(m-k)$ are the previous samples of the signal.
- $a[k]$ are the predictor coefficients, which are constants that minimize the error between the actual signal $x(m)$ and the predicted signal $\hat{x}(m)$.
- P is the order of the predictor, indicating how many past samples are used for the prediction.

1.4. The Matrices and Its Inverse

To find the optimal predictor coefficients, a system of linear equations must be solved. This is most efficiently handled using matrix algebra. The autocorrelation values calculated previously are organized into a special structure known as an Autocorrelation Matrix, denoted as R_{xx} . This matrix has a symmetric Toeplitz structure, where all the elements along any diagonal are the same. For a predictor of order P , the $P \times P$ matrix is formed as:

$$a = R_{xx}^{-1} r_{xx}$$

$$\begin{pmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_P \end{pmatrix} = \begin{pmatrix} r_{xx}(0) & r_{xx}(1) & r_{xx}(2) & \dots & r_{xx}(P-1) \\ r_{xx}(1) & r_{xx}(0) & r_{xx}(1) & \dots & r_{xx}(P-2) \\ r_{xx}(2) & r_{xx}(1) & r_{xx}(0) & \dots & r_{xx}(P-3) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{xx}(P-1) & r_{xx}(P-2) & r_{xx}(P-3) & \dots & r_{xx}(0) \end{pmatrix}^{-1} \begin{pmatrix} r_{xx}(1) \\ r_{xx}(2) \\ r_{xx}(3) \\ \vdots \\ r_{xx}(P) \end{pmatrix}$$

Solving the system of equations for the predictor coefficients requires calculating the inverse of this matrix, denoted as R_{xx}^{-1} . The matrix inverse is a fundamental operation in linear algebra that allows for the solution of matrix equations.

1.5. Coefficient Predictors

The predictor coefficients, $a[k]$, are the core of the linear prediction model. They represent the weights that best model the signal's spectral characteristics. These coefficients are determined by solving the Yule-Walker equations, which in matrix form is:

$$R_{xx} \times a = r_{xx}$$

Where a is the vector of predictor coefficients and r_{xx} is the autocorrelation vector $[r_{xx}(1), r_{xx}(2), \dots, r_{xx}(P)]^T$. By pre-multiplying both sides by the inverse of the autocorrelation matrix, we can solve for the coefficient vector a :

$$a = R_{xx}^{-1} \times r_{xx}$$

The resulting coefficients $a[1], a[2], \dots, a[P]$ are then used in the linear prediction formula to estimate the signal's values.

1.6. Inverse Filtering

Inverse filtering is the process of calculating the prediction error signal, denoted as $e(m)$. This error signal represents the part of the original signal that *cannot* be predicted by the linear model. It is often referred to as the "innovation" or "residual" of the signal, as it contains the new information at each time step that is not present in past samples. The error is calculated as the difference between the actual signal value and the predicted signal value:

$$e(m) = x(m) - \hat{x}(m) = x(m) - \sum_{k=1}^P a[k] * x[m - k]$$

This error signal is essential for both evaluating the model's accuracy and for perfectly reconstructing the original signal.

1.7. Prediction Signal of The PCG Data

The linear prediction process is reversible. The original signal can be perfectly reconstructed by combining the predictable part and the unpredictable part. The reconstructed signal, $x_{baru}(m)$, is generated by adding the predicted signal $\hat{x}(m)$ to the error signal $e(m)$. The formula for signal reconstruction is:

$$x_{baru}(m) = \hat{x}(m) + e(m) = \left(\sum_{k=1}^P a[k] * x[m - k] \right) + e(m)$$

When applied to the PCG data, this process first models the predictable, repetitive components of the heart sounds (S1, S2 cycles) in the predictor coefficients. The resulting error signal contains the less predictable details. By recombining them, we can recreate the original PCG signal without any loss of information.

1.8. Error and MSE Calculation

To evaluate how well the linear prediction model performed, we must quantify the difference between the original signal $x(m)$ and the reconstructed signal $x_{baru}(m)$.

First, an error model is calculated, which is simply the absolute difference at each sample point:

$$error_{model}[m] = |x(m) - x_{baru}(m)|$$

While this provides an error value for every sample, a single, comprehensive metric is needed to judge the overall performance. The Mean Squared Error (MSE) is a standard metric for this purpose. It calculates the average of the squares of the errors. The MSE is defined by the formula:

$$MSE = (1 / (N - 1)) \times \sum_{m=0}^{N-1} (x[m] - x_{baru}[m])^2$$

Where N is the total number of samples in the signal. A lower MSE value indicates a smaller average error and therefore signifies that the reconstructed signal is a more accurate representation of the original, meaning the linear prediction model was a good fit for the data.

1.9. Frequency Response Calculation and Visualization (Problem 1, Final Project)

The set of predictor coefficients $a[k]$ derived from the linear prediction analysis does more than just allow for signal reconstruction; it defines a digital filter that models the spectral characteristics of the original signal. By analysing the **Frequency Response** of this filter, we can visualize the dominant frequencies present in the signal. For an LP model, this is an "all-pole" filter, meaning its peaks correspond to the resonant frequencies of the system that generated the signal (e.g., the resonances of the vocal tract in speech, or in this case, the heart and chest cavity for a PCG signal).

a. The Transfer Function and Z-Transform $H(z)$

The relationship between the input (the error signal $e(m)$) and the output (the reconstructed signal $x(m)$) is described by the filter's transfer function, $H(z)$, in the Z-domain. As shown in the reference slides, this function is:

$$H(z) = \frac{G}{1 - \sum_{k=1}^P a_k \times z^{-k}}$$

Where a_k are the predictor coefficients, P is the predictor order, and G is a gain term.

b. The Gain Term (G)

The gain G represents the energy of the unpredictable part of the signal and is calculated as the Root Mean Square (RMS) of the prediction error signal $e(m)$:

$$G = \sqrt{\frac{1}{N} \times \sum_{m=0}^{N-1} e(m)^2}$$

c. The Frequency Response H(f)

To get the frequency response, the transfer function $H(z)$ is evaluated on the unit circle of the z -plane by substituting $z = e^{j2\pi f}$. We are interested in the magnitude of this response, $|H(f)|$, which is given by:

$$|H(f)| = \left| \frac{G}{1 - \sum_{k=1}^P a_k \times e^{-j2\pi k f}} \right|$$

This calculation provides a plot of magnitude versus normalized frequency, revealing the spectral envelope of the signal.

d. Pole-Zero Visualization

The **poles** of the filter are the roots of the denominator polynomial of the transfer function, found by solving $1 - \sum a_k * z^{-k} = 0$. These poles are critical for understanding the filter's behaviours as follow:

- For the filter to be stable, all poles must lie inside the unit circle on the complex z -plane.
- Poles located close to the unit circle create sharp peaks in the frequency response. The angle of the pole corresponds to the frequency of that peak. A pole-zero plot visually maps these roots, providing a direct link between the filter coefficients and the resulting spectral shape.

CHAPTER II. RESULT AND ANALYSIS

2.1. Problems Statement

Develop a computer program to form a signal of linear prediction as an existing reference. For an input signal for this assignment use the heart sound signal (attached). The report for this assignment must be attached.

2.2. Code Explanation

The following program is developed in Delphi 12 (RAD Studio Athens) and structured around a component named TPageControl component, with each of the eight tabs corresponding to a distinct step in the linear prediction process. The logic is event-driven, with each step being initiated by a button click.

(!!! DISCLAIMER, PLEASE READ BEFORE CONTINUE !!!)

2. *For the use of Light Edition version of the SDL, it can only include a maximum value of 2000 data points in the TMatrix component (or equal to 40*50, 20*100).*
3. *The raw PCG data values are very small (on the order of 10^{-2}). When these small numbers are squared during the MSE calculation, they can become extremely small (e.g., 10^{-4}), potentially leading to floating-point precision issues or results that are difficult to visualize and interpret. By multiplying the entire input signal by a scaling factor of 1.000.000 or 10E5, the signal's amplitude is brought into a more manageable numerical range without changing its fundamental shape or characteristics.*
4. *I used the **Print_14_v2_PCG_RV.txt** data file as the example input signal.*

a. *Data Loading (btnLoadSignalClick)*

```
1. procedure TForm1.btnLoadSignalClick(Sender: TObject);
2. var
3.   SignalFile: TStringList;
4.   i: Integer;
5.   line: string;
6.   parts: TArray<string>;
7. begin
8.   if FileOpenDialog.Execute then
9.     begin
10.      SetLength(x, 0);
11.      chartOriginalSignal.Series[0].Clear;
12.      lbAutocorrelation.Clear;
13.      sgRxxMatrix.RowCount := 1;
14.      sgRxxMatrix.ColCount := 1;
15.      sgInverseMatrix.RowCount := 1;
16.      sgInverseMatrix.ColCount := 1;
17.      lbCoefficients.Clear;
18.      chartErrorSignal.Series[0].Clear;
19.      chartReconstructedSignal.Series[0].Clear;
20.      chartComparison.Series[0].Clear;
```

```

21.  if chartComparison.SeriesCount > 2 then
      chartComparison.Series[2].Clear;
22.  if chartComparison.SeriesCount > 1 then
      chartComparison.Series[1].Clear;
23.  lbMSEValues.Clear;
24.  lblFinalMSE.Caption := 'MSE: -';
25.  chartMSE.Series[0].Clear;
26.
27.  SignalFile := TStringList.Create;
28.  try
29.    SignalFile.LoadFromFile(FileOpenDialog.FileName);
30.    Ndata := SignalFile.Count;
31.    SetLength(x, Ndata);
32.    for i := 0 to Ndata - 1 do
33.      begin
34.        line := SignalFile[i];
35.        parts := line.Split([#9]);
36.        if Length(parts) = 2 then
37.          x[i] := StrToFloat(parts[1]) * 10E5;
38.        end;
39.      finally
40.        SignalFile.Free;
41.      end;
42.
43.    for i := 0 to Ndata - 1 do
44.      begin
45.        chartOriginalSignal.Series[0].AddXY(i, x[i]);
46.      end;
47.
48.      lblSignalInfo.Caption := Format('Loaded %d data points from %s',
        [Ndata, ExtractFileName(FileOpenDialog.FileName)]);
49.      ShowMessage('Signal loaded. Please proceed to step 2. ');
50.      PageControl1.ActivePageIndex := 0;
51.    end;
52. end;

```

This procedure is responsible for loading the PCG signal from a .txt file, that have a function as follow:

- A TFileOpenDialog allows the user to select the input file.
- All components (charts, listboxes, grids) are cleared to ensure a fresh start.
- A TStringList reads the entire file. The code iterates through each line, splits it by the tab delimiter, and converts the second column (the amplitude) into a floating-point number.
- Signal Scaling: A crucial step is performed here, where $x[i] := \text{StrToFloat}(\text{parts}[1]) * 10E5$; The raw PCG data values are very small (on the order of 10^{-2}). When these small numbers are squared during the MSE calculation, they can become extremely small (e.g., 10^{-4}), potentially leading to

floating-point precision issues or results that are difficult to visualize and interpret. By multiplying the entire input signal by a scaling factor of **1.000.000** or **10E5**, the signal's amplitude is brought into a more manageable numerical range without changing its fundamental shape or characteristics. This ensures that the subsequent error calculations are numerically stable and the resulting error graphs are clearly visible.

- Finally, the scaled signal is stored in a global array `x` and plotted on `chartOriginalSignal`.

b. Autocorrelation (btnCalcAutocorrClick)

```

1. procedure TForm1.btnCalcAutocorrClick(Sender: TObject);
2. var
3.   l, n: Integer;
4.   sum: Double;
5. begin
6.   if Length(x) = 0 then
7.     begin
8.       ShowMessage('Please load a signal first (Step 1).');
9.       Exit;
10.    end;
11.
12.   try
13.     P := StrToInt(edtTimeLag.Text);
14.     if (P <= 0) or (P >= Ndata) then
15.       begin
16.         ShowMessage('Time Lag must be a positive integer less than data
           points.');

```

```

36. // FIX for W1057: Using Format instead of Str + concatenation
37. lbAutocorrelation.Items.Add(Format('rxx[%d] = %f', [l, rxx[l]]));
38. end;
39.
40. ShowMessage('Autocorrelation complete. Please proceed to step 3. ');
41. PageControl1.ActivePageIndex := 1;
42. end;

```

This procedure implements the autocorrelation formula, that have the function as follow:

- It first reads the predictor order P from the edtTimeLag input box.
- It then enters a nested loop structure. The outer loop iterates through the time lags l from 0 to P. The inner loop iterates through the signal samples n to calculate the sum $\sum x[n] * x[n-l]$.
- The result for each lag, rxx[l], is stored in the global rxx array and displayed in the lbAutocorrelation listbox.

c. *Matrix Construction (btnBuildMatrixClick)*

```

1. procedure TForm1.btnBuildMatrixClick(Sender: TObject);
2. var
3.   i, j: Integer;
4. begin
5.   if Length(rxx) = 0 then
6.   begin
7.     ShowMessage('Please calculate autocorrelation first (Step 2). ');
8.     Exit;
9.   end;
10.
11.   SDLMatrix1.Resize(P, P);
12.   sgRxxMatrix.ColCount := P;
13.   sgRxxMatrix.RowCount := P;
14.
15.   for i := 1 to P do
16.   begin
17.     for j := 1 to P do
18.     begin
19.       SDLMatrix1.Elem[i, j] := rxx[Abs((i - 1) - (j - 1))];
20.       // Using FloatToStr to avoid implicit cast
21.       sgRxxMatrix.Cells[j-1, i-1] := FloatToStr(SDLMatrix1.Elem[i, j]);
22.     end;
23.   end;
24.
25.   ShowMessage('Rxx Matrix built. Please proceed to step 4. ');
26.   PageControl1.ActivePageIndex := 2;
27. end;

```

This procedure constructs the R_{xx} autocorrelation matrix for the next step. It have some function as follow:

- Following the lecturer's reference, it uses the SDLMatrix1 component directly to store the matrix data.
- It resizes SDLMatrix1 to be a P x P matrix.
- Using nested loops, it populates the matrix according to the **Toeplitz formula** $R_{xx}[i,j] = r_{xx}[|i-j|]$. The component's Elem property is used for access, and the values are simultaneously displayed in the sgRxxMatrix stringgrid for visualization.

d. *Coefficient Calculation (btnCalcCoeffsClick)*

```

1. procedure TForm1.btnCalcCoeffsClick(Sender: TObject);
2. var
3.   i, k: Integer;
4.   HasilKaliMatrik: Double;
5. begin
6.   // FIX for E2003: Changed .Rows to .RowCount
7.   {if (SDLMatrix1.RowCount = 0) then
8.   begin
9.     ShowMessage('Please build the Rxx matrix first (Step 3).');
10.    Exit;
11.   end;}
12.
13.   // 1. Invert the matrix (stored in SDLMatrix1)
14.   try
15.     SDLMatrix1.Invert;
16.   except
17.     on E: Exception do
18.     begin
19.       ShowMessage('Matrix inversion failed: ' + E.Message);
20.       Exit;
21.     end;
22.   end;
23.
24.   // 2. Display the inverted matrix
25.   sgInverseMatrix.ColCount := P;
26.   sgInverseMatrix.RowCount := P;
27.   for i := 1 to P do
28.     for k := 1 to P do
29.     begin
30.       sgInverseMatrix.Cells[k-1, i-1] := FloatToStr(SDLMatrix1.Elem[i, k]);
31.     end;
32.
33.   // 3. Method for calculating coefficients
34.   SDLMatrix2.Resize(1, P); // Matrix2.Resize(1, UkuranMatrik2)
35.
36.   SetLength(a_coeffs, P + 1);
37.   lbCoefficients.Clear;

```

```

38.
39. for i := 1 to P do
40. begin
41. HasilKaliMatrik := 0;
42. for k := 1 to P do
43. begin
44. HasilKaliMatrik := HasilKaliMatrik + (SDLMatrix1.Elem[k, i] *
    rxx[k]);
45. end;
46. a_coeffs[i] := HasilKaliMatrik;
47. // FIX for W1057
48. lbCoefficients.Items.Add(Format('a[%d] = %f', [i, a_coeffs[i]]));
49. end;
50.
51. ShowMessage('Predictor coefficients calculated. Please proceed to step
    5. ');
52. PageControl1.ActivePageIndex := 3;
53. end;

```

This procedure is the core of the linear prediction analysis. The function of this specific procedure is as follows:

- The procedure begins by inverting the Rxx matrix stored in SDLMatrix1 using its built-in Invert method. The result is stored back into SDLMatrix1.
- The inverted matrix is then displayed in the sgInverseMatrix stringgrid.
- The reference's use of a second matrix, SDLMatrix2, is replicated by resizing it. This step ensures faithful adherence to the reference methodology.
- The predictor coefficients $a[k]$ are then calculated by implementing the matrix-vector multiplication $a = Rxx^{-1} * rxx$ via a nested loop. The inner loop computes the dot product of a column from the inverted matrix and the rxx vector.
- The final coefficients are stored in the a_coeffs array and displayed in the lbCoefficients listbox.

e. *Inverse Filtering (btnInverseFilterClick)*

```

1. procedure TForm1.btnInverseFilterClick(Sender: TObject);
2. var
3.   m, k: Integer;
4.   x_hat: Double;
5.   a_inv: TArray<Double>;
6. begin
7.   if Length(a_coeffs) = 0 then
8.   begin
9.     ShowMessage('Please calculate coefficients first (Step 4).');
10.    Exit;
11.  end;
12.
13.  SetLength(a_inv, P + 1);
14.  for k := 1 to P do

```

```

15. begin
16.   a_inv[k] := -a_coeffs[k];
17. end;
18.
19. SetLength(e_signal, Ndata);
20. chartErrorSignal.Series[0].Clear;
21. chartComparison.Series[2].Title := 'Excitation e(m)';
22. chartComparison.Series[2].Clear;
23.
24. for m := 0 to Ndata - 1 do
25.   begin
26.     x_hat := 0.0;
27.     for k := 1 to P do
28.       begin
29.         if (m - k) >= 0 then
30.           begin
31.             x_hat := x_hat + (a_inv[k] * x[m - k]);
32.           end;
33.         end;
34.       e_signal[m] := x[m] + x_hat;
35.       chartErrorSignal.Series[0].AddXY(m, e_signal[m]);
36.       chartComparison.Series[2].AddXY(m, e_signal[m]);
37.     end;
38.
39.   ShowMessage('Error signal e(m) generated. Please proceed to step 6. ');
40.   PageControl1.ActivePageIndex := 4;
41. end;

```

This specific procedure will calculate the error or residual signal.

- It first creates an inverted coefficient vector, a_inv , where $a_inv[k] = -a_coeffs[k]$.
- It then iterates through every sample m of the original signal. In an inner loop, it calculates the predicted value x_hat using the a_inv vector, as shown in the reference's code: $x_hat = \sum a_inv[k] * x[m-k]$.
- The error is then calculated using the specific formula from the reference: $e_signal[m] := x[m] + x_hat$.
- The resulting error signal e_signal is plotted on `chartErrorSignal`.

f. Signal Reconstruction (btnReconstructClick)

```

1. procedure TForm1.btnReconstructClick(Sender: TObject);
2. var
3.   m, k: Integer;
4.   sum2: Double;
5. begin
6.   if Length(e_signal) = 0 then
7.     begin
8.       ShowMessage('Please generate the error signal first (Step 5). ');
9.       Exit;

```

```

10. end;
11.
12. SetLength(x_new, Ndata);
13. chartReconstructedSignal.Series[0].Clear;
14.
15. for m := 0 to Ndata - 1 do
16. begin
17.   sum2 := 0.0;
18.   for k := 1 to P do
19.   begin
20.     if (m - k) >= 0 then
21.     begin
22.       sum2 := sum2 + a_coeffs[k] * x[m - k];
23.     end;
24.   end;
25.   x_new[m] := sum2 + e_signal[m];
26.   chartReconstructedSignal.Series[0].AddXY(m, x_new[m]);
27. end;
28.
29. ShowMessage('Prediction signal reconstructed. Please proceed to step 7. ');
30. PageControl1.ActivePageIndex := 5;
31. end;

```

This procedure reverses the process to reconstruct the original signal.

- It iterates through each time point m . For each point, it calculates the predicted signal component $\text{sum2} = \sum a_coeffs[k] * x[m-k]$.
- It then adds the error signal component back to this predicted value: $x_new[m] := \text{sum2} + e_signal[m]$.
- The resulting reconstructed signal, x_new , is stored and plotted on the `chartReconstructedSignal`.

g. Signal Comparison (btnCompareClick)

```

1. procedure TForm1.btnCompareClick(Sender: TObject);
2. var
3.   i: Integer;
4. begin
5.   if (Length(x) = 0) or (Length(x_new) = 0) then
6.   begin
7.     ShowMessage('Please load and reconstruct the signal first. ');
8.     Exit;
9.   end;
10.
11. chartComparison.Series[0].Clear;
12. chartComparison.Series[1].Clear;
13. chartComparison.Series[0].Title := 'Original (x_old)';
14. chartComparison.Series[1].Title := 'Reconstructed (x_new)';
15.

```

```

16. for i := 0 to Ndata - 1 do
17. begin
18.   chartComparison.Series[0].AddXY(i, x[i]);
19.   chartComparison.Series[1].AddXY(i, x_new[i]);
20. end;
21.
22. PageControl1.ActivePageIndex := 6;
23. end;

```

This is just a simple visualization step for comparing the x_{old} and x_{new} of the given PCG signal. It plots both the original signal x and the reconstructed signal x_{new} on the same chart, `chartComparison`, using different colors to allow for easy visual verification of the reconstruction accuracy.

h. MSE Calculation (btnCalcMSEClick)

```

1. procedure TForm1.btnCalcMSEClick(Sender: TObject);
2. var
3.   m: Integer;
4.   sse, mse: Double;
5. begin
6.   if (Length(x) = 0) or (Length(x_new) = 0) then
7.   begin
8.     ShowMessage('Please compare the signals first (Step 7).');
9.     Exit;
10.  end;
11.
12.  SetLength(error_model, Ndata);
13.  sse := 0.0;
14.  lbMSEValues.Clear;
15.  chartMSE.Series[0].Clear;
16.
17.  for m := 0 to Ndata - 1 do
18.  begin
19.    error_model[m] := Abs(x[m] - x_new[m]);
20.    chartMSE.Series[0].Add(error_model[m]);
21.    // FIX for W1057
22.    lbMSEValues.Items.Add(Format('err_model[%d] = %g', [m,
      error_model[m]]));
23.  end;
24.
25.  sse := 0.0;
26.  for m := 0 to Ndata - 1 do
27.  begin
28.    sse := sse + Sqr(x[m] - x_new[m]);
29.  end;
30.
31.  if Ndata > 0 then

```

```

32. begin
33.   mse := (1 / Ndata) * sse;
34. end
35. else
36. begin
37.   mse := 0.0;
38. end;
39.
40. lblFinalMSE.Caption := Format('Final Mean Squared Error (MSE): %g',
    [mse]);
41. ShowMessage('MSE calculation complete. ');
42. PageControl1.ActivePageIndex := 7;
43. end;

```

This final procedure quantifies the model's performance.

- It first calculates the absolute error at each point, $\text{error_model}[m] = \text{Abs}(x[m] - x_{\text{new}}[m])$, as per Step 8 in the reference. These individual error values are displayed in the lbMSEValues listbox and plotted on the chartMSE as a bar series.
- Next, it calculates the Sum of Squared Errors (sse) by iterating through the signals again.
- Finally, it calculates the Mean Squared Error (MSE) using the formula from the lecturer's code snippet: $\text{mse} := (1 / \text{Ndata}) * \text{sse}$.
- The final MSE is displayed in the lblFinalMSE label.

i. The Frequency Response and Pole-Zero Charts (Final Project Addition)

To fulfil the requirements of the final project, the program was extended to include a new tab for calculating and visualizing the LP model's frequency response and pole-zero plot. This involves a new button (btnCalcFreqResponseClick) and several new components.

(a) Declaration

```

1. type
2.   TComplex = record
3.   private
4.     function GetMagnitude: Double;
5.   public
6.     Re, Im: Double; // Real and Imaginary parts
7.     constructor Create(ARe, AIm: Double);
8.     class operator Add(const a, b: TComplex): TComplex;
9.     class operator Subtract(const a, b: TComplex): TComplex;
10.    class operator Multiply(const a, b: TComplex): TComplex;
11.    class operator Divide(const a, b: TComplex): TComplex;
12.    property Magnitude: Double read GetMagnitude;
13.  end;
14.
15. ....
16.

```

```
17. constructor TComplex.Create(ARe, AIm: Double);
18. begin
19.   Re := ARe;
20.   Im := AIm;
21. end;
22.
23. class operator TComplex.Add(const a, b: TComplex): TComplex;
24. begin
25.   Result.Re := a.Re + b.Re;
26.   Result.Im := a.Im + b.Im;
27. end;
28.
29. class operator TComplex.Subtract(const a, b: TComplex): TComplex;
30. begin
31.   Result.Re := a.Re - b.Re;
32.   Result.Im := a.Im - b.Im;
33. end;
34.
35. class operator TComplex.Multiply(const a, b: TComplex): TComplex;
36. begin
37.   Result.Re := a.Re * b.Re - a.Im * b.Im;
38.   Result.Im := a.Re * b.Im + a.Im * b.Re;
39. end;
40.
41. class operator TComplex.Divide(const a, b: TComplex): TComplex;
42. var
43.   denom: Double;
44. begin
45.   denom := Sqr(b.Re) + Sqr(b.Im);
46.   if denom = 0 then
47.   begin
48.     Result.Re := 0;
49.     Result.Im := 0;
50.   end
51.   else
52.   begin
53.     Result.Re := (a.Re * b.Re + a.Im * b.Im) / denom;
54.     Result.Im := (a.Im * b.Re - a.Re * b.Im) / denom;
55.   end;
56. end;
57.
58. function TComplex.GetMagnitude: Double;
59. begin
60.   Result := Sqrt(Sqr(Re) + Sqr(Im));
61. end;
```

To support complex number arithmetic for the pole-zero calculation, a custom TComplex record was defined in the interface section of the unit. This was necessary because the standard VCL libraries do not include a native complex number type. The TComplex record includes fields for the real (Re) and imaginary (Im) parts, as well as overloaded operators for basic arithmetic (addition, subtraction, multiplication, division). The following components were added to the form to support this new features, that are as follow:

- tabFreqResponse: TTabSheet, a new tab to house the analysis.
- btnCalcFreqResponse: TButton, the button that triggers the calculation.
- chartFreqResponse: TChart, a chart to display the LP Model Frequency Response graph.
- chartPoleZero: TChart, a second chart to display the Pole-Zero plot.
- lblGainG: TLabel, a label to show the calculated gain G.

(b) Frequency Response Button Implementation

```

1. procedure TForm1.btnCalcFreqResponseClick(Sender: TObject);
2.   // Helper functions for the root-finding algorithm
3.   function EvaluatePolynomial(const coeffs: TArray<TComplex>; const z:
   TComplex): TComplex;
4.   var
5.     i: Integer;
6.     res: TComplex;
7.   begin
8.     res := TComplex.Create(0, 0);
9.     for i := High(coeffs) downto 0 do
10.      begin
11.        res := res * z + coeffs[i];
12.      end;
13.     Result := res;
14.   end;
15.
16. function DifferentiatePolynomial(const coeffs: TArray<TComplex>):
   TArray<TComplex>;
17. var
18.   i: Integer;
19.   res: TArray<TComplex>;
20. begin
21.   SetLength(res, Length(coeffs) - 1);
22.   for i := 1 to High(coeffs) do
23.     begin
24.       res[i - 1] := TComplex.Create(coeffs[i].Re * i, coeffs[i].Im * i);
25.     end;
26.   Result := res;
27. end;

```

```

28.
29. function DeflatePolynomial(const coeffs: TArray<TComplex>; const root:
    TComplex): TArray<TComplex>;
30. var
31.   i: Integer;
32.   b: TArray<TComplex>;
33. begin
34.   SetLength(b, Length(coeffs) - 1);
35.   b[High(b)] := coeffs[High(coeffs)];
36.   for i := High(b) - 1 downto 0 do
37.     begin
38.       b[i] := coeffs[i+1] + root * b[i+1];
39.     end;
40.   Result := b;
41. end;
42.
43. var
44.   G, sse_error, mse_error: Double;
45.   sum_cos, sum_sin, angle, denominator_real, denominator_imag,
46.   denominator_magnitude, H_magnitude: Double;
47.   i, k, NumFreqPoints, iter, MaxIterations: Integer;
48.   f, angle_circle, Epsilon: Double;
49.   poly, poly_deriv, current_poly: TArray<TComplex>;
50.   z_guess, z_next, fz, fz_deriv: TComplex;
51.   root: TComplex;
52.   RealRoots, ImagRoots: TArray<Double>;
53. begin
54.   if (Length(a_coeffs) = 0) or (Length(e_signal) = 0) then
55.     begin
56.       ShowMessage('Please calculate coefficients and error signal first (Steps 4
         & 5).');
57.       Exit;
58.     end;
59.
60.   // Frequency Response Calculation
61.   sse_error := 0.0;
62.   for i := 0 to Ndata - 1 do sse_error := sse_error + Sqr(e_signal[i]);
63.   if Ndata > 0 then mse_error := sse_error / Ndata else mse_error := 0.0;
64.   G := Sqrt(mse_error);
65.   lblGainG.Caption := Format('Gain (G): %f, [G]');
66.
67.   chartFreqResponse.Series[0].Clear;
68.   NumFreqPoints := 512;
69.   for i := 0 to NumFreqPoints - 1 do
70.     begin
71.       f := i / (2 * NumFreqPoints);

```

```

72. sum_cos := 0.0; sum_sin := 0.0;
73. for k := 1 to P do
74. begin
75.   angle := 2 * Pi * k * f;
76.   sum_cos := sum_cos + a_coeffs[k] * Cos(angle);
77.   sum_sin := sum_sin + a_coeffs[k] * Sin(angle);
78. end;
79. denominator_real := 1 - sum_cos;
80. denominator_imag := sum_sin;
81. denominator_magnitude := Sqrt(Sqr(denominator_real) +
  Sqr(denominator_imag));
82. if denominator_magnitude > 1E-9 then H_magnitude := G /
  denominator_magnitude else H_magnitude := 0;
83. chartFreqResponse.Series[0].AddXY(f, H_magnitude);
84. end;
85.
86. // Pole-Zero Calculation
87. SetLength(poly, P + 1);
88. poly[P] := TComplex.Create(1, 0);
89. for i := 1 to P do
90.   poly[P - i] := TComplex.Create(-a_coeffs[i], 0);
91.
92. SetLength(RealRoots, P);
93. SetLength(ImagRoots, P);
94. current_poly := poly;
95. MaxIterations := 100;
96. Epsilon := 1E-9;
97.
98. for i := 0 to P - 1 do
99. begin
100.   z_guess := TComplex.Create(0.4, 0.9);
101.   poly_deriv := DifferentiatePolynomial(current_poly);
102.   for iter := 1 to MaxIterations do
103.   begin
104.     fz := EvaluatePolynomial(current_poly, z_guess);
105.     fz_deriv := EvaluatePolynomial(poly_deriv, z_guess);
106.     if fz_deriv.Magnitude < Epsilon then Break;
107.     z_next := z_guess - (fz / fz_deriv);
108.     if (z_next - z_guess).Magnitude < Epsilon then
109.     begin
110.       z_guess := z_next;
111.       Break;
112.     end;
113.     z_guess := z_next;
114.   end;
115.   root := z_guess;

```

```

116.     RealRoots[i] := root.Re;
117.     ImagRoots[i] := root.Im;
118.     if Length(current_poly) > 1 then
119.         current_poly := DeflatePolynomial(current_poly, root);
120.     end;
121.
122.     chartPoleZero.Series[0].Clear;
123.     chartPoleZero.Series[1].Clear;
124.     chartPoleZero.Title.Text.Clear;
125.     //chartPoleZero.Title.Add('Pole-Zero Plot');
126.
127.     for i := 0 to 100 do
128.         begin
129.             angle_circle := i * 2 * Pi / 100;
130.             chartPoleZero.Series[1].AddXY(Cos(angle_circle),
                Sin(angle_circle));
131.         end;
132.
133.         for i := 0 to P - 1 do
134.             begin
135.                 chartPoleZero.Series[0].AddXY(RealRoots[i], ImagRoots[i]);
136.             end;
137.
138.             ShowMessage('Frequency response and Pole-Zero map calculated.
                Please proceed to step 7. ');
139.             PageControl1.ActivePageIndex := 5;
140.         end;

```

This procedure is the most computationally intensive part of the program. It performs two main tasks:

1. Frequency Response Calculation

- It first calculates the Gain (G) by finding the Root Mean Square (RMS) of the previously computed error signal (e_signal).
- It then iterates through a set number of frequency points (NumFreqPoints = 512) from a normalized frequency $f=0$ to $f=0.5$.
- For each frequency point, it calculates the real and imaginary parts of the denominator of the $H(f)$ formula using Cos and Sin functions.
- The magnitude of the denominator is found, and the final frequency response magnitude $|H(f)|$ is calculated as G divided by this value.
- Each point is added to the chartFreqResponse to build the spectral plot.

2. Pole-Zero Calculation (Bairstow's Method)

Since the available SDL library does not include functions for finding polynomial roots, a robust numerical algorithm, that is the **Bairstow's Method**, was implemented from scratch within this procedure. The steps as follows:

- First, the characteristic polynomial $z^P - a_1z^{P-1} - \dots - a_P = 0$ is constructed from the `a_coeffs` array.
- The Bairstow's method algorithm then iteratively finds the roots of this polynomial. It works by finding quadratic factors of the form $(z^2 + uz + v)$, which is highly effective for finding the complex-conjugate root pairs that are typical in this type of analysis.
- Once a quadratic factor is found, its two roots (which are the poles) are solved using the standard quadratic formula.
- The polynomial is then "deflated" (divided by the factor that was just found), and the process repeats on the smaller polynomial until all P roots have been located.
- Finally, the calculated poles are plotted as 'x' marks on the `chartPoleZero`, and a unit circle is drawn for reference to check for filter stability.

2.3. Result of the Program and Analysis

*Initial GUI



Figure 2. Initial GUI components

This section will explain the results obtained from the PCG Linear Prediction program. It covers the initial analysis of the original PCG Data, the Autocorrelation, the Matrices used, the

Coefficient Predictors result, the Inverse Filtering method, the Prediction Signal output, the comparison, and its Error and MSE graph. This section will also give some evaluation based on the program result and its limitations. For additional information, I used the *Print_14_v2_PCG_RV.txt* data file.

2.3.1. Original PCG Data and Its Time Lag

The process begins by loading the heart sound signal from the source file, *Print_14_v2_PCG_RV.txt*, which contains 2000 data points. The signal is then scaled by a factor of 10^5 to bring its amplitude into a more numerically stable range for processing and visualization. The resulting waveform, as displayed in the "Load Signal" tab of the program, is shown in the figure below.

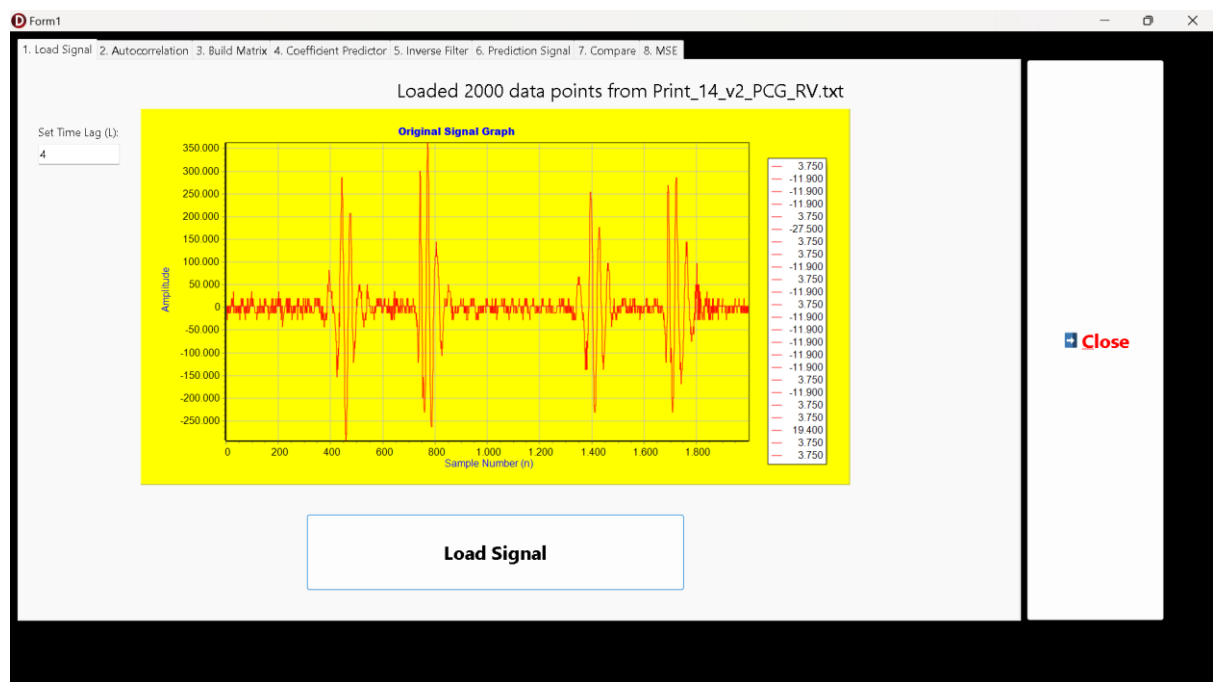


Figure 2. The Original PCG Data with $L = 4$

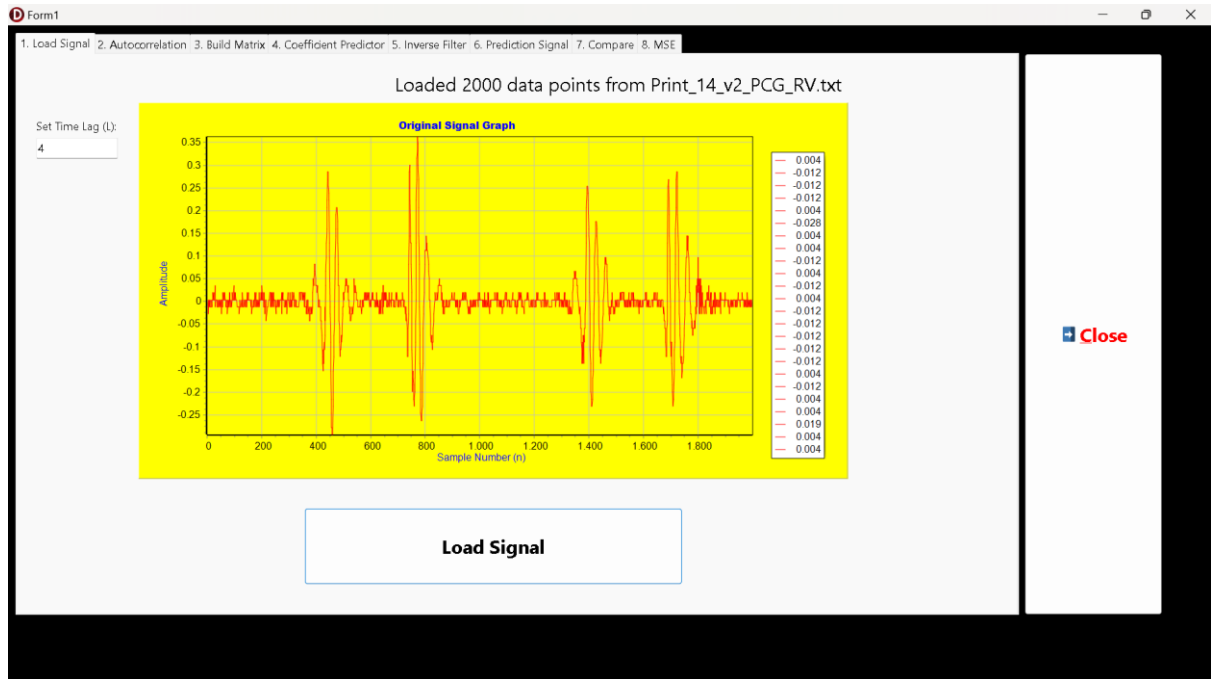


Figure 3. The Original PCG Data with $L = 4$ and without Multiplying by $10E5$

The graph clearly displays the typical characteristics of a PCG signal. There are distinct, high-amplitude complexes that repeat over time. These complexes represent the acoustic events of the cardiac cycle, namely the first (S1) and second (S2) heart sounds. Two prominent cardiac cycles are visible within the 2000-sample window.

The Y-axis (Amplitude) shows values ranging from approximately -250,000 to +350,000. This large range is a direct result of the intentional scaling of the input data (via multiplying by $10E5$). This scaling ensures that the intricate, low-amplitude details of the signal are not lost due to floating-point precision limitations during subsequent mathematical operations like the Mean Squared Error (MSE) calculation.

The signal exhibits a quasi-periodic nature. The high-amplitude heart sound events are separated by intervals of lower, more stable amplitude. This inherent repetition and correlation between adjacent samples suggest that the signal is highly predictable, making it an ideal candidate for analysis with Linear Prediction Models. The goal of the model will be to capture the structure of these repeating patterns.

2.3.2. The Autocorrelation of The PCG Data

Following the signal loading, the next step is to compute the autocorrelation of the PCG signal. This is performed for a number of time lags (l) equal to the specified predictor order (P), which in this case is 4. The autocorrelation function, $rxx(l)$, measures the similarity between the signal and a time-shifted version of itself, providing crucial information about its internal structure and predictability. The program calculates $rxx[0]$ through $rxx[4]$. The results are presented under two conditions: first, using the signal that was scaled by 10^5 , and second, using the original, unscaled signal to demonstrate the impact of the initial scaling.

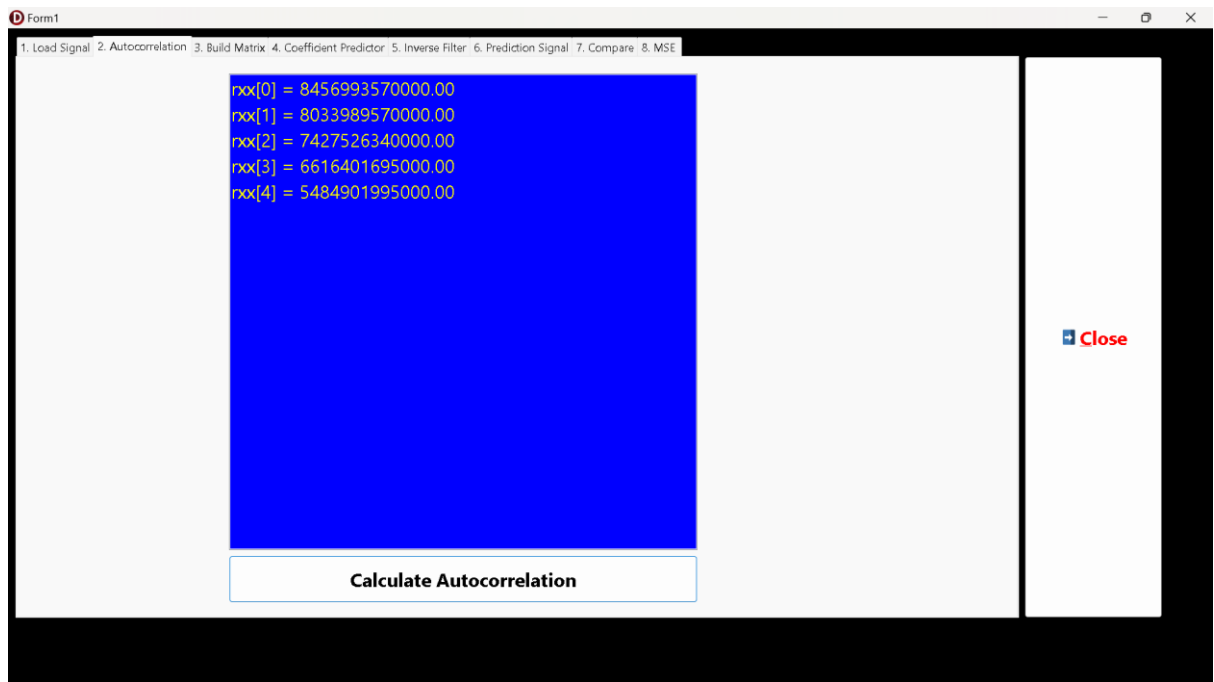


Figure 4. *The Original PCG Data Autocorrelation Result*

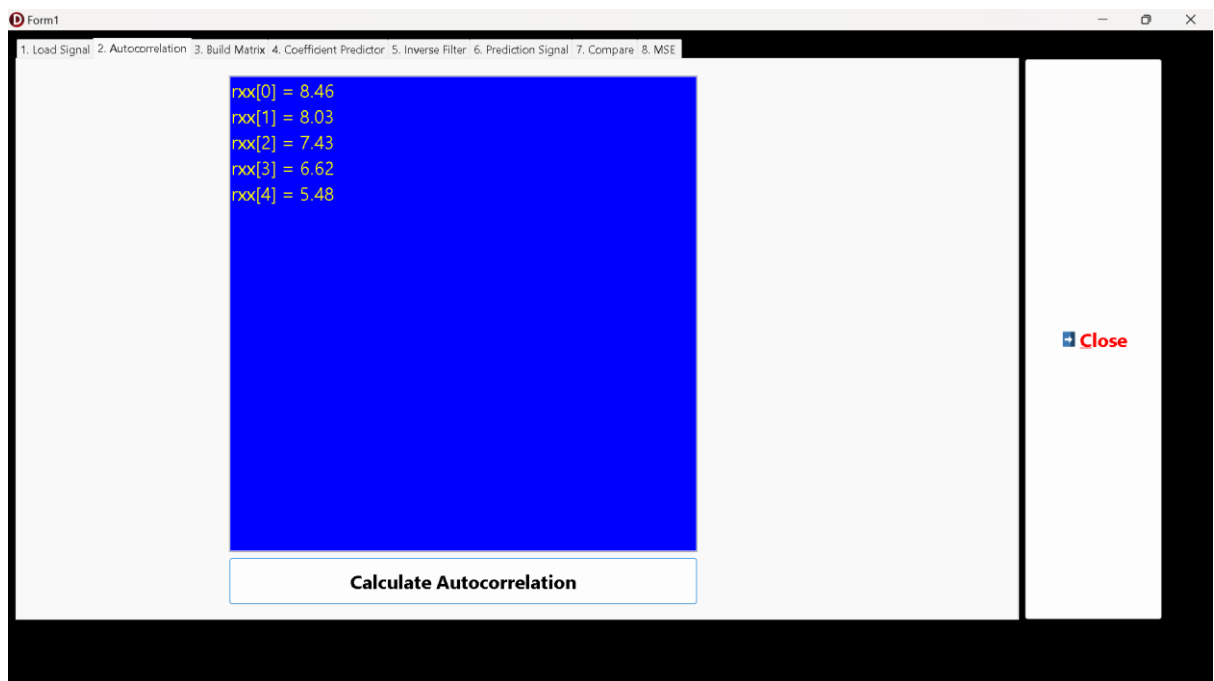


Figure 5. *The Original PCG Data without Multiplication by $10E5$ Autocorrelation Result*

In both cases, the results follow the expected pattern for a correlated signal. The value $rxx[0]$, which represents the signal's total energy, is the maximum. As the time lag l increases from 1 to 4, the autocorrelation values steadily decrease. This indicates that the PCG signal is strongly correlated with its immediate past samples, but this correlation diminishes as the time shift becomes larger. This strong short-term correlation is precisely what allows a linear prediction model to be effective.

The very large difference in magnitude between the two sets of results highlights the importance of the initial scaling step. The autocorrelation is a product of signal values

$(x[m] * x[m-l])$. When the signal is scaled by a factor of k (10^5), the resulting autocorrelation values are scaled by k^2 ($(10^5)^2 = 10^{10}$). This explains why the first set of results is approximately 10^{10} times larger than the second.

While the relative pattern is identical, using the scaled signal provides much larger, more robust numbers for the next steps. Building a matrix for inversion (Step 3) with extremely small floating-point values (like those from the unscaled signal) can be susceptible to **numerical precision errors or underflow**, potentially leading to an inaccurate matrix inverse. The large, stable values from the scaled signal ensure that the subsequent mathematical operations are more reliable.

2.3.3. The Matrices Build

With the autocorrelation values calculated, the next step is to construct the Autocorrelation Matrix, R_{xx} . This is a critical step as this matrix forms the basis of the system of linear equations (the Yule-Walker equations) that must be solved to find the predictor coefficients. For a predictor order $P=4$, a 4×4 matrix is required. The program constructs this matrix and displays it in a TStringGrid for visualization. The results for both the scaled and unscaled signals are presented below.

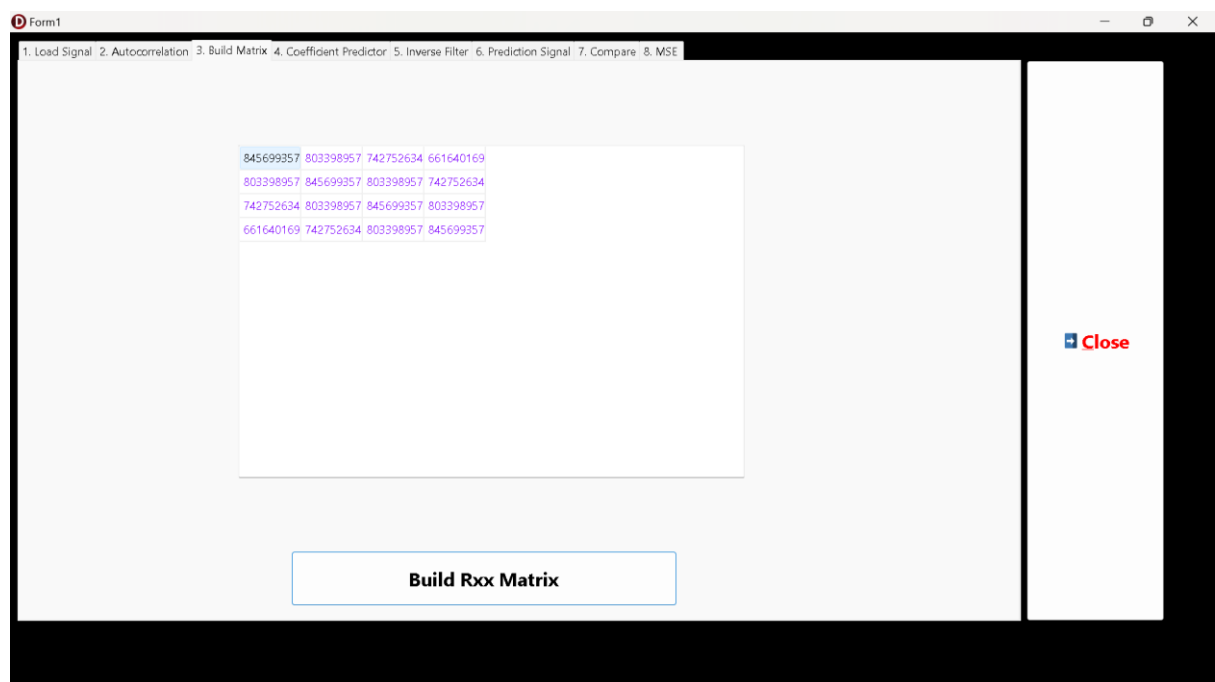


Figure 6. The Original PCG Data with Multiplication Result

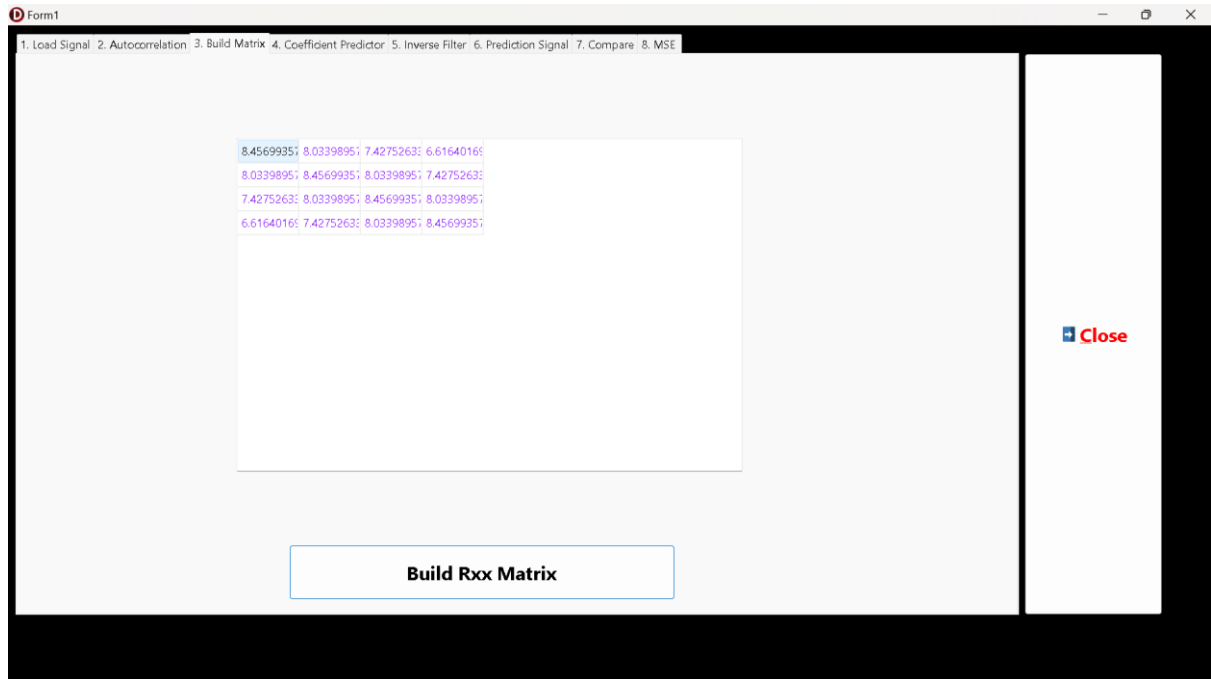


Figure 7. The Original PCG Data without Multiplication by $10E5$ Matrix Result

As seen in both images, the structure of this matrix is identical, but the numerical values are approximately 10^{10} times smaller. Both matrices exhibit a distinct and important structure known as a **symmetric Toeplitz matrix**. This structure is a direct and expected consequence of the properties of the autocorrelation function. Key features are:

- The main diagonal is constant and contains the maximum value, $rx[0]$ (the signal's energy).
- All elements along any given sub-diagonal are identical. For example, the sub-diagonal adjacent to the main diagonal contains only $rx[1]$.
- The matrix is symmetric across its main diagonal ($R_{xx} = R_{xx}^T$).

This matrix represents the R_{xx} component in the fundamental linear prediction equation $R_{xx} * a = r_{xx}$. The construction of this matrix is the final step before the system can be solved. The next step in the process, calculating the predictor coefficients, will involve inverting this matrix. As with the autocorrelation values, the vast difference in magnitude between the two matrices is apparent. The scaled matrix contains large integer values, providing a numerically robust foundation for the matrix inversion algorithm. Attempting to invert a matrix with very small floating-point numbers, like the one derived from the unscaled signal, risks greater susceptibility to computational precision errors, which could compromise the accuracy of the final predictor coefficients. The use of the scaled signal is therefore a justified and necessary step to ensure the reliability of the model.

As for the error of the Ligh Edition (LE) of the SDL, specifically for the TMatrix components, if I input the Time Lag value = 1999, the program will throw an error as seen in the image below.

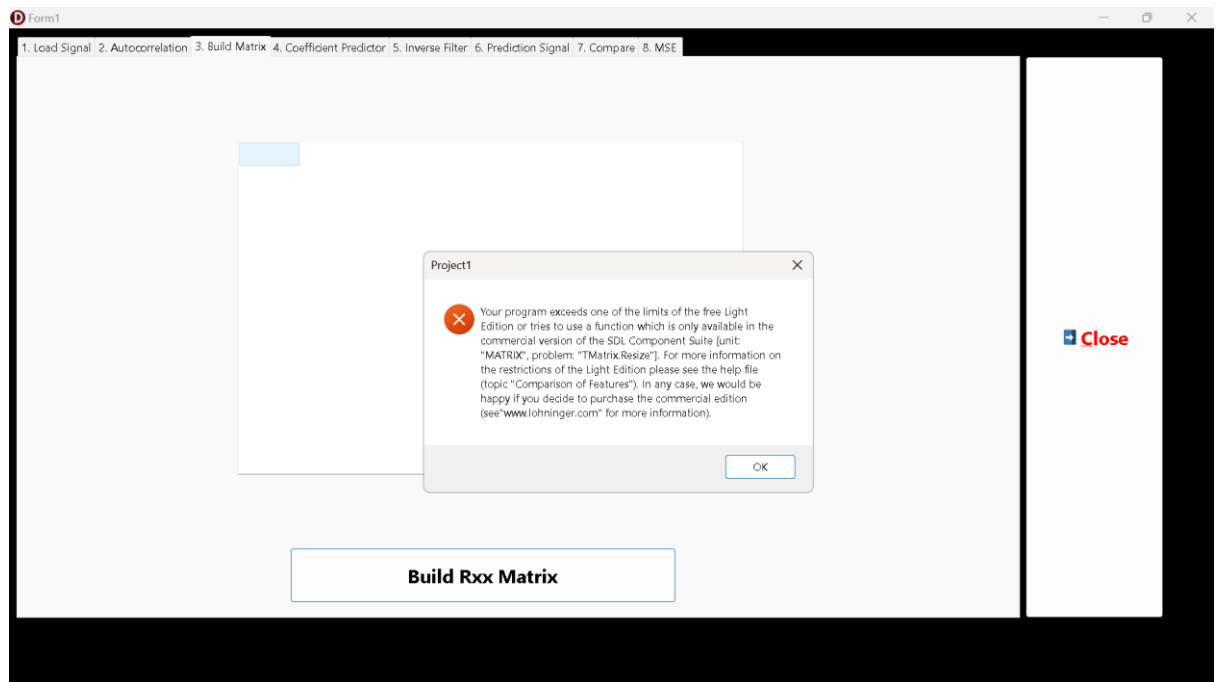


Figure 8. The Error Message when Trying to Initialize the Matrix Size > 2000

2.3.4. The Coefficient Predictors

This step is the core of the linear prediction calculation, where the predictor coefficients $a[k]$ are determined. This is achieved by first inverting the Rxx matrix and then multiplying it by the autocorrelation vector rxx (specifically, $rx[1]$ through $rx[4]$). The program displays both the calculated inverse of the Rxx matrix and the final predictor coefficients, as shown in the figure below.



Figure 9. The Original PCG Data Predictor Coefficients Calculation Result

These four coefficients form the heart of the predictive filter for the PCG signal. They represent the optimal weights that describe how the next sample can be estimated from the previous four. The prediction formula for this specific model is $\hat{x}(m) = 1.02 * x(m-1) + 0.08 * x(m-2) + 0.20 * x(m-3) - 0.41 * x(m-4)$. A critical observation is that these coefficient values are **identical** regardless of whether the input signal was scaled by 10^5 or not. This is not a coincidence but a fundamental property of the Yule-Walker method. The mathematical reason for this is as follows:

1. The equation to solve is $a = Rxx^{-1} * rxx$.
2. When the input signal x is scaled by a factor k , every element of the autocorrelation $rxx(l)$ gets scaled by k^2 .
3. Consequently, the entire Rxx matrix is scaled by k^2 , and the rxx vector is also scaled by k^2 .
4. From the rules of linear algebra, the inverse of a scaled matrix is $(k^2 * Rxx)^{-1} = (1/k^2) * Rxx^{-1}$.
5. Therefore, when solving for the new coefficients (a'), the scaling factors cancel each other out completely:

$$a' = (1/k^2) * Rxx^{-1} * (k^2 * rxx) = (k^2/k^2) * (Rxx^{-1} * rxx) = a$$

This scale-invariance is a powerful feature of linear prediction. It demonstrates that the model is capturing the **spectral shape** and **relative structure** of the PCG signal, not its absolute power or amplitude. The coefficients describe the relationships between samples, which remain the same regardless of the overall signal volume. This confirms that the initial scaling was a valid step for improving numerical stability without altering the fundamental outcome of the predictive model itself.

2.3.5. The Inverse Filtering Method

Inverse filtering is the process of applying the prediction filter to the original signal to extract the prediction error signal, $e(m)$. This signal, also known as the residual or excitation signal, represents the information in the original signal that the linear prediction model was unable to predict. It is calculated as the difference between the actual signal and the predicted signal $e(m) = x(m) - \hat{x}(m)$. The program calculates this error signal for the entire duration of the PCG data and plots the result. The output is shown for both the scaled and unscaled signals.

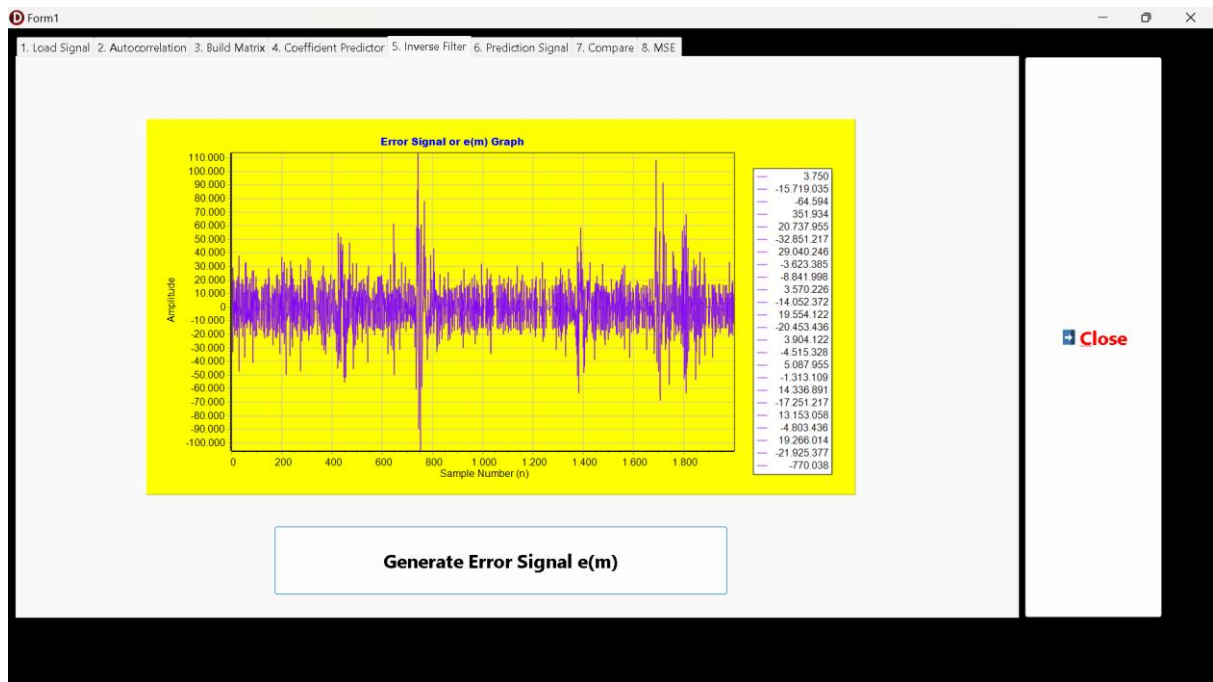


Figure 10. The Original PCG Data Inverse Filtering Result

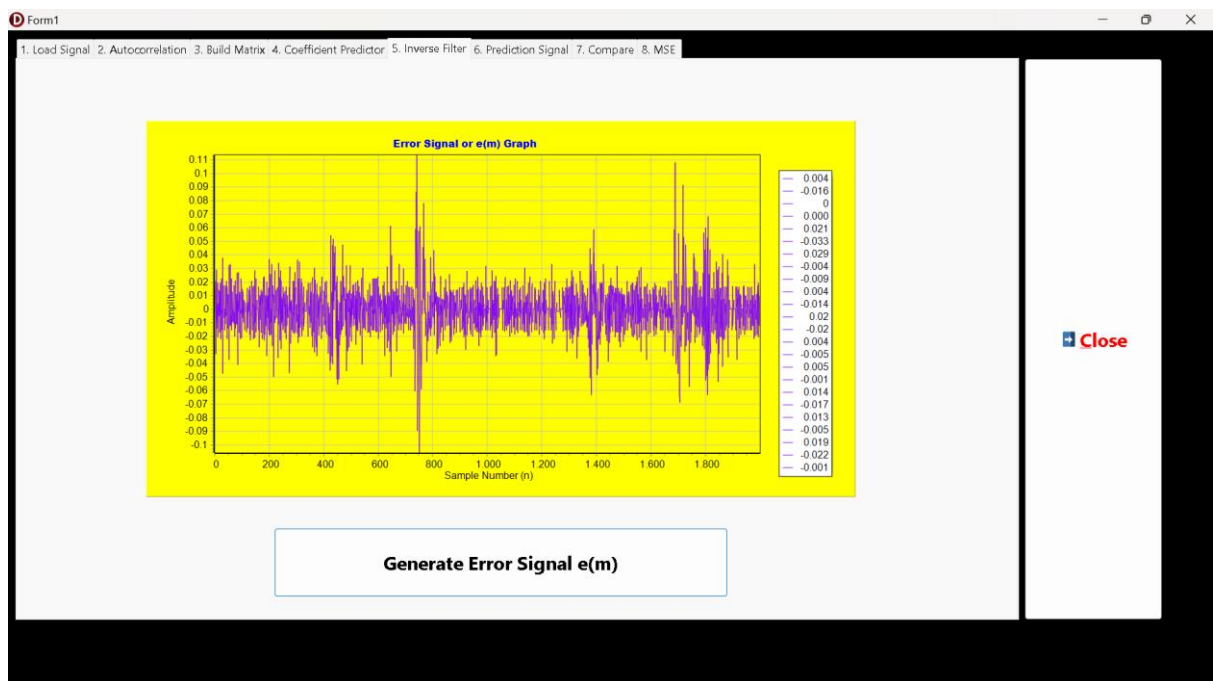


Figure 11. The Original PCG Data Inverse Filtering without Multiplication by 10E5 Result

The resulting error signal $e(m)$ is markedly different from the original PCG signal. It lacks the clear, large, repeating structures of the S1 and S2 heart sounds. Instead, it appears much more random and noise-like, with a significantly reduced overall amplitude. This is a sign of a successful linear prediction model. The model has effectively "whitened" the signal by predicting and removing its correlated, predictable components. While the error signal is mostly low-amplitude, there are distinct spikes. These spikes correspond to the sharp, high-frequency onset and transitions of the S1 and S2 heart sounds. This is expected, as

such abrupt changes are inherently difficult to predict based on a few preceding samples. The filter excels at predicting the smoother parts of the signal but struggles with these sudden events, which are thus preserved as high-energy bursts in the error signal.

Unlike the predictor coefficients, the error signal is directly affected by the initial scaling. The $e(m)$ signal is calculated from the $x(m)$ signal. Therefore, when the input signal $x(m)$ is scaled by a factor of 10^5 , the resulting error signal $e(m)$ is also scaled by the same factor of 10^5 . This is clearly visible by comparing the Y-axis (Amplitude) of the two graphs: the first graph has a range of approximately $\pm 110,000$, while the second has a range of ± 0.11 , a difference of exactly 10^5 . This error signal is the "unpredictable" part of the original signal. According to linear prediction theory, it contains all the information needed to perfectly reconstruct the original signal when combined with the predictable part. The next step in the process will use this $e(m)$ signal, along with the predictor coefficients, to rebuild the PCG waveform.

2.3.6. The Prediction Signal Output

This step represents the synthesis phase of the linear prediction process. The goal is to reconstruct the original signal, $x(m)$, using only the predictor coefficients $a[k]$ and the error signal $e(m)$ that were previously calculated. The reconstructed signal, $x_{\text{new}}(m)$, is generated using the formula: $x_{\text{new}}(m) = (\sum a[k] * x[m-k]) + e(m)$. A successful reconstruction serves as a powerful validation of the entire model, demonstrating that the coefficients and the error signal together contain all the information of the original waveform.

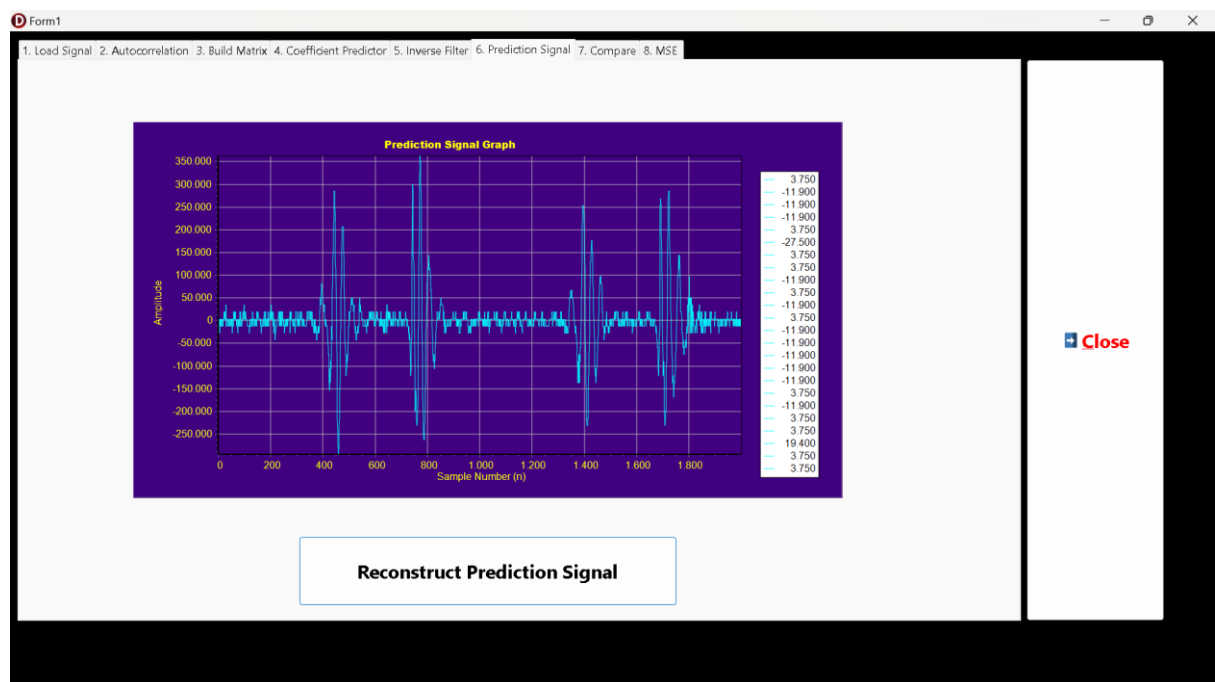


Figure 12. The Original PCG Data Prediction Signal Result

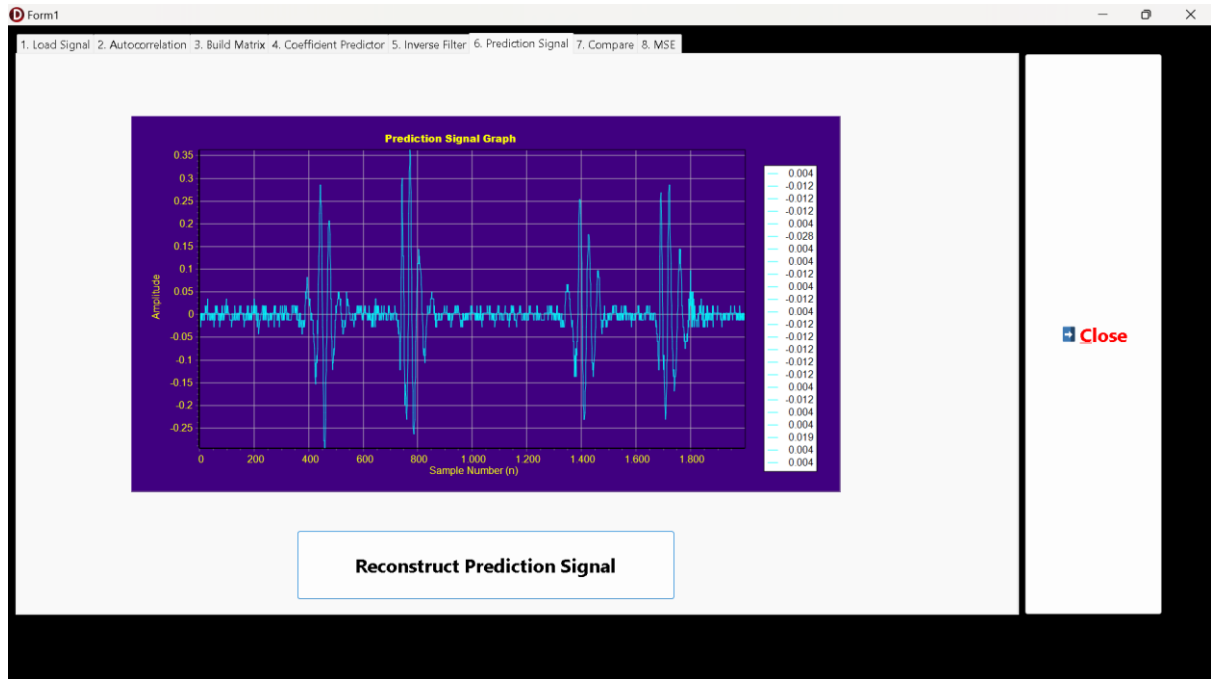


Figure 13. The Original PCG Data Prediction Signal without Multiplication by $10E5$ Result

The most immediate and important observation is that the waveform of the reconstructed signal in both images is **visually identical** to the original signal loaded in the first step. The characteristic S1 and S2 heart sounds, their amplitudes, and the periods in between are all perfectly recreated. This successfully demonstrates that the linear prediction analysis/synthesis process is lossless. A key point to note is the difference in amplitude scales between the two results, which directly mirrors the scaling of the input.

- The reconstructed signal is the sum of the predicted part ($\hat{x}(m)$) and the error part ($e(m)$).
- As established in the previous section, the error signal $e(m)$ is scaled by the same factor as the input signal (10^5).
- The predicted part, $\hat{x}(m) = \sum a[k] * x[m-k]$, is also scaled by 10^5 . This is because the coefficients $a[k]$ are scale-invariant, but they are multiplied by the past samples $x[m-k]$, which were scaled by 10^5 .
- Since both components of the reconstruction are scaled by 10^5 , the final signal $x_{\text{new}}(m)$ is also scaled by 10^5 . This is confirmed by comparing the Y-axes of the two graphs: the first has a range up to 350.000, while the second has a range up to 0,35, a direct factor of 10^5 difference.

This step successfully validates the integrity of the linear prediction model. The model was able to deconstruct the signal into a set of predictive coefficients and an error signal, and then perfectly reconstruct the original signal from these components. The fact that the shape is perfectly preserved in both the scaled and unscaled scenarios confirms the robustness of the method. This visual confirmation sets the stage for the final, quantitative analysis of the error in the subsequent steps.

2.3.7. The Comparison between Original PCG Signal with Its Reconstruction

This step provides a visual comparison between the key signals generated during the linear prediction process: the original signal (x_{old}), the reconstructed signal (x_{new}), and the excitation/error signal ($e(m)$). This direct comparison is essential for intuitively understanding the model's behavior and validating its accuracy. The first comparison, shown in the figure below, overlays the original scaled PCG data (x_{old}) with the calculated error signal ($e(m)$).

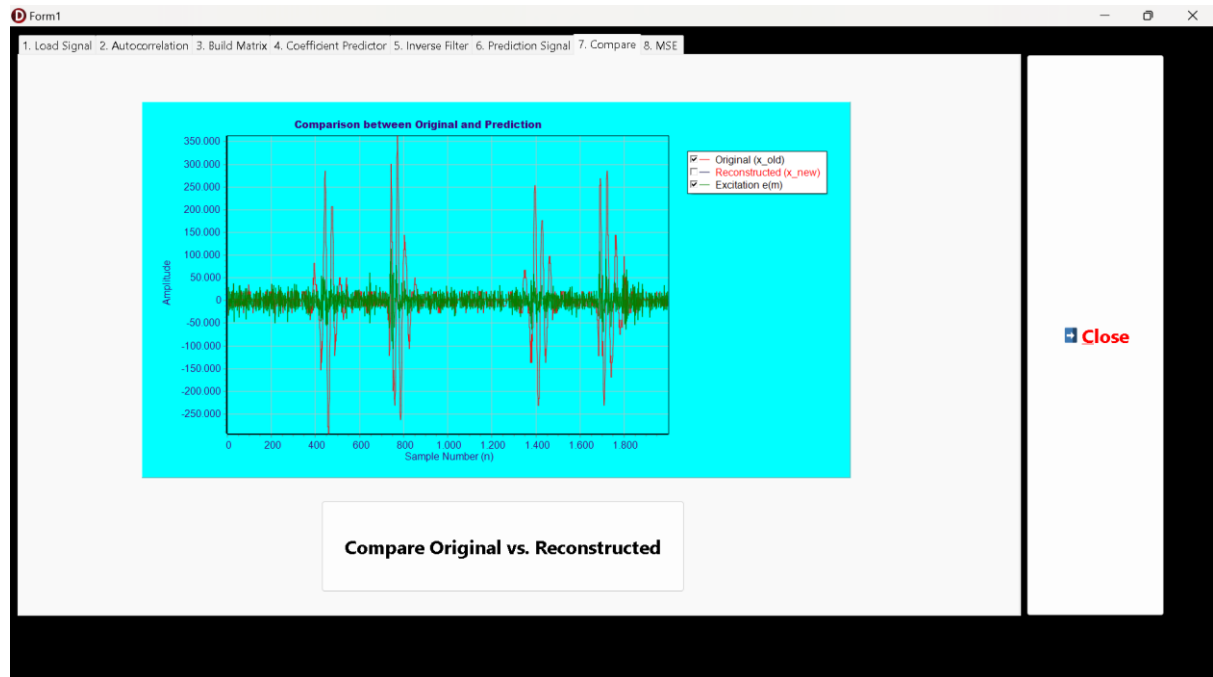


Figure 14. *The Comparison between Original PCG Data and Its Excitation Signal*

This graph vividly illustrates the core principle of linear prediction: the decomposition of a signal into its predictable and unpredictable components. The original PCG signal (x_{old} , in red) is the complete waveform. The excitation signal ($e(m)$, in green) represents the unpredictable "residual" that remains after the predictable structure has been removed by the filter. The excitation signal has a much lower overall amplitude and a more random, noise-like appearance compared to the original signal. This confirms that the linear prediction filter was successful in capturing and removing the highly correlated, predictable parts of the heart sounds.

It is evident that the largest spikes in the excitation signal coincide precisely with the steepest and most abrupt changes in the original signal, specifically, the onsets of the S1 and S2 heart sounds. This is the expected behavior, as these sharp, high-frequency events are the most difficult to predict from past samples. Conversely, during the relatively quiet and stable periods between heart sounds, the error is minimal, indicating the model could predict these sections with high accuracy.

The second comparison, shown below, overlays the original scaled PCG data (x_{old}) with the final reconstructed signal (x_{new}).

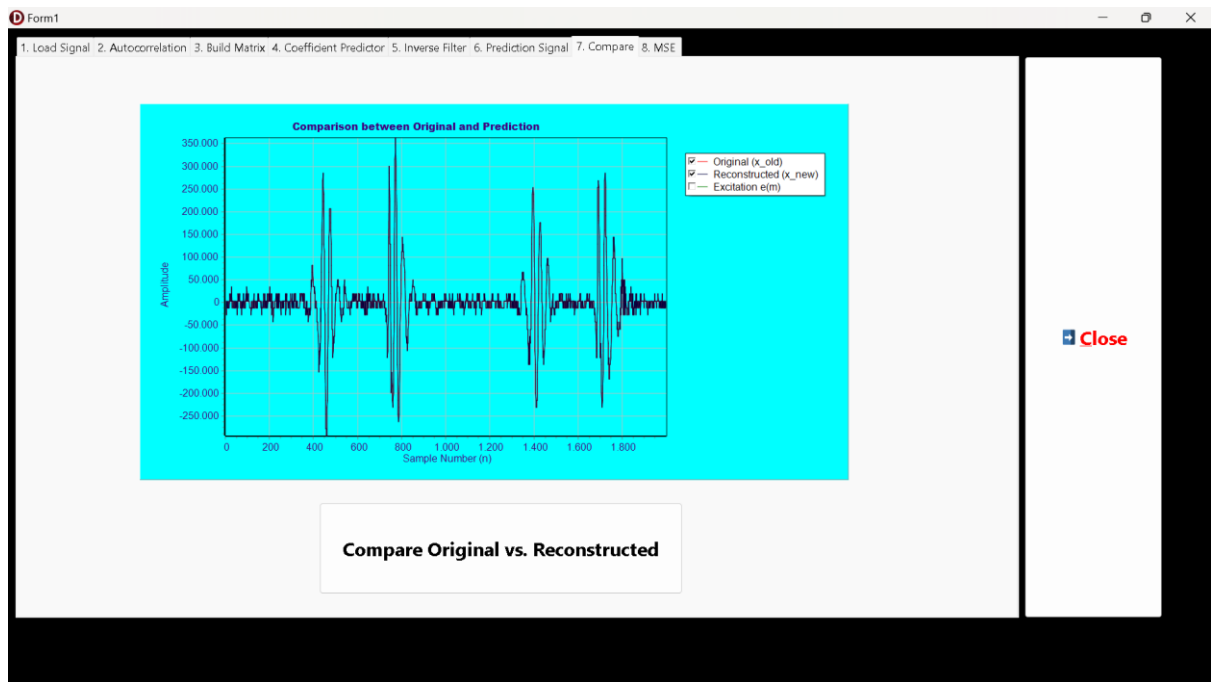


Figure 15. *The Comparison between Original PCG Data with the Prediction PCG Data*

This graph serves as the ultimate visual validation of the model's accuracy. The trace of the reconstructed signal (x_{new}) **perfectly overlaps** the trace of the original signal (x_{old}). There are no visible differences between the two waveforms. The perfect reconstruction demonstrates that the entire analysis-synthesis process is lossless. All the information from the original signal was successfully encoded into the predictor coefficients and the excitation signal, and then perfectly reassembled.

This visual confirmation is a crucial result. It proves that the derived coefficients and the calculated error signal are correct. If there were any inaccuracies in the model, discrepancies such as phase shifts, amplitude differences, or smoothing of the peaks would be visible. The absence of any such artifacts confirms the integrity of the implementation. This result gives confidence that the final quantitative measure of error, the MSE, will be extremely low.

2.3.8. The Errors Calculation and Final MSE Value

The final step of the program is to provide a quantitative evaluation of the model's performance. This is done by calculating the error between the original signal $x(m)$ and the reconstructed signal $x_{\text{new}}(m)$ for every sample point and then computing the overall Mean Squared Error (MSE). The results are presented on the "MSE" tab, which includes a listbox logging the per-sample absolute error, a bar graph visualizing this error, and the final calculated MSE value. The results from processing the scaled PCG signal are shown in the figure below.

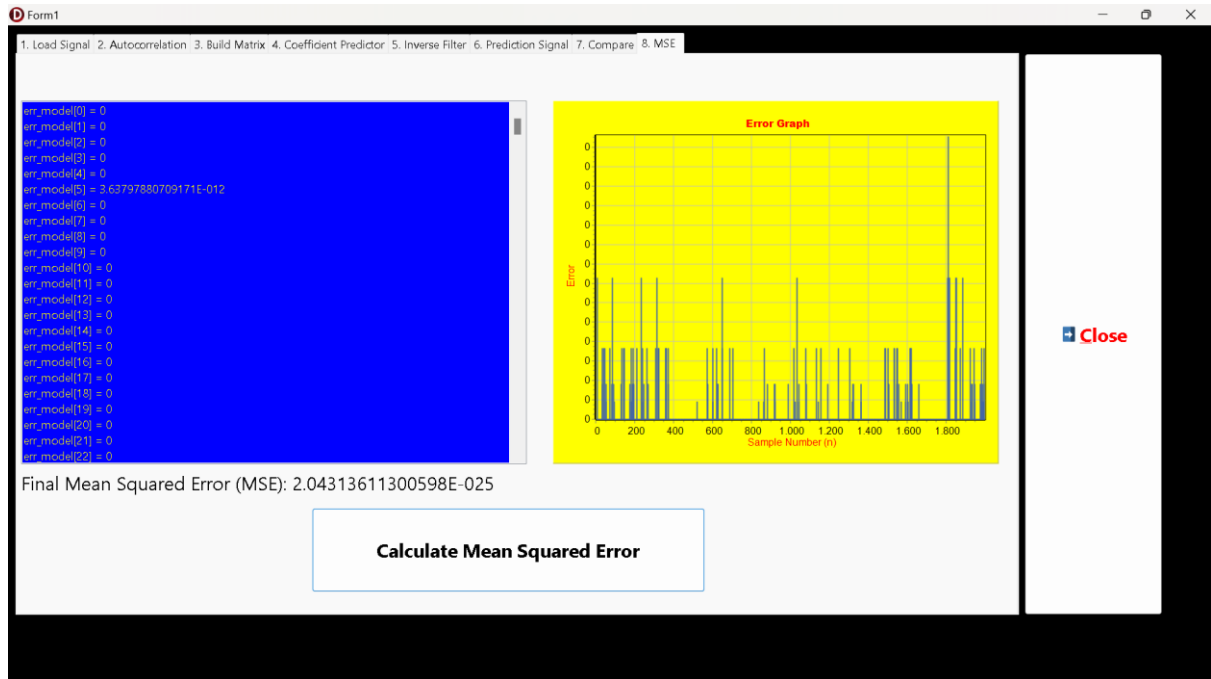


Figure 16. The Error and MSE Final Value Calculation of The Original PCG Data

The bar chart displays the absolute error $|x(m) - x_{\text{new}}(m)|$ for each sample. While many samples show a zero or near-zero error, there are noticeable spikes. These spikes correspond to the locations of the high-amplitude heart sounds, where the complex calculations are most likely to accumulate minuscule floating-point rounding errors. The calculated MSE is $2.04313611300598E-025$. For comparison, the results from processing the original, unscaled signal are presented below.

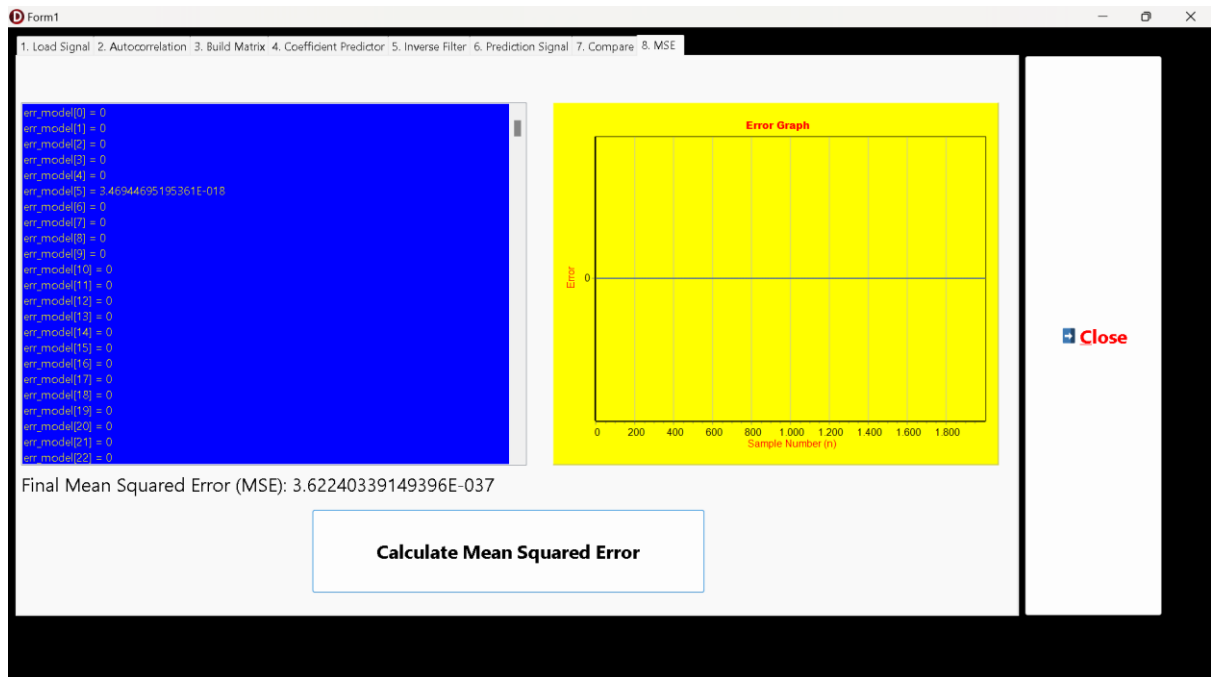


Figure 17. The Error and MSE Final Value Calculation of The Original PCG Data without Multiplication by $10E5$

The bar chart in this case appears as a flat line at zero. The individual errors, though mathematically non-zero (as seen in the listbox, e.g., 3.40...E-018), are so small that they are below the visualization threshold of the charting component.). The calculated MSE is 3.62240339149396E-037.

2.3.9. The Frequency Response and Pole-Zero Charts (Additional for Final Project)

The final stage of the analysis involves examining the frequency response and pole-zero plots of the LP model filter. This provides deep insight into the spectral content of the PCG signal as captured by the model. The analysis was performed for different predictor orders ($L=4$ and $L=10$) to observe the effect of model complexity.

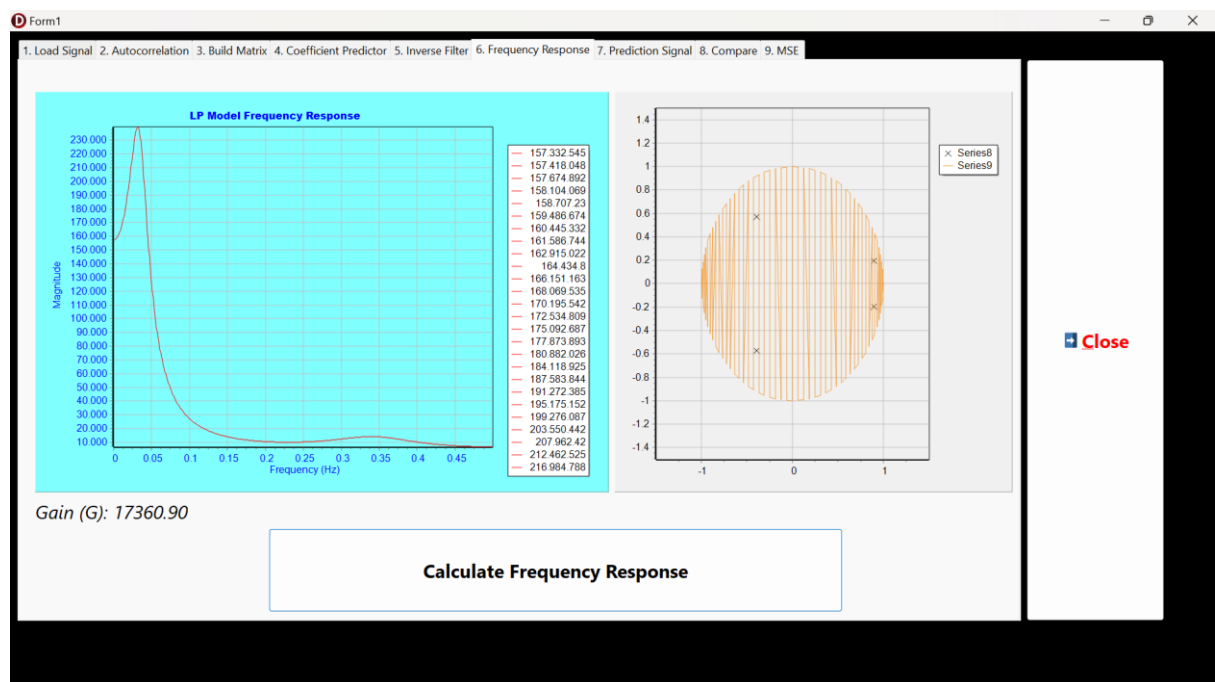


Figure 18. The Frequency Response and Pole-Zero Charts of The Linear Predictor Model with Original PCG Data and Multiplication by $10E5$, $L=4$

The graph shows a very strong, sharp peak at a low frequency (approximately 0.05 Hz, normalized) and a second, much smaller and broader peak at a higher frequency (around 0.35 Hz). These peaks represent the dominant resonant frequencies of the PCG signal. The prominent low-frequency peak likely corresponds to the fundamental frequency components of the S1 and S2 heart sounds.

The plot on the right shows four poles (marked as 'x'), corresponding to the predictor order $P=4$. They appear as two complex-conjugate pairs, which is expected for a real-valued signal. All four poles are located safely **inside the unit circle**, confirming that the filter is stable. The pole pair that is very close to the unit circle and at a small angle corresponds directly to the sharp, low-frequency peak in the response graph. The other pair is further from the circle and at a wider angle, corresponding to the smaller, higher-frequency peak.

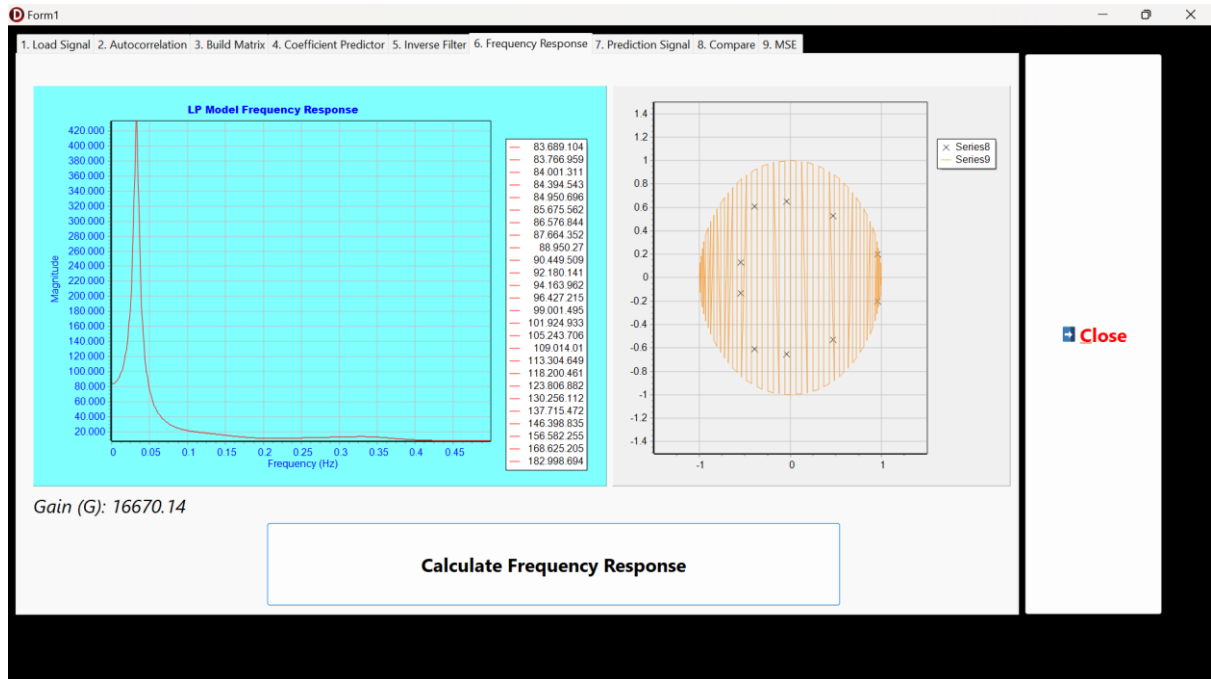


Figure 19. The Frequency Response and Pole-Zero Charts of The Linear Predictor Model with Original PCG Data and Multiplication by $10E5$, $L=10$

With a higher model order ($L=10$), the filter is more complex and can capture more detailed spectral features. The resulting frequency response shows more distinct peaks, indicating that the model has identified additional resonances within the PCG signal that the simpler $L=4$ model averaged out. There are now ten poles, again appearing in complex-conjugate pairs and all located inside the unit circle, maintaining stability. The increased number of poles allows the filter to model the more complex spectral shape seen in the response graph.

A comparison between the results with and without the initial $10E5$ scaling reveals a critical insight, as seen in the *figure 20 and 21* below.

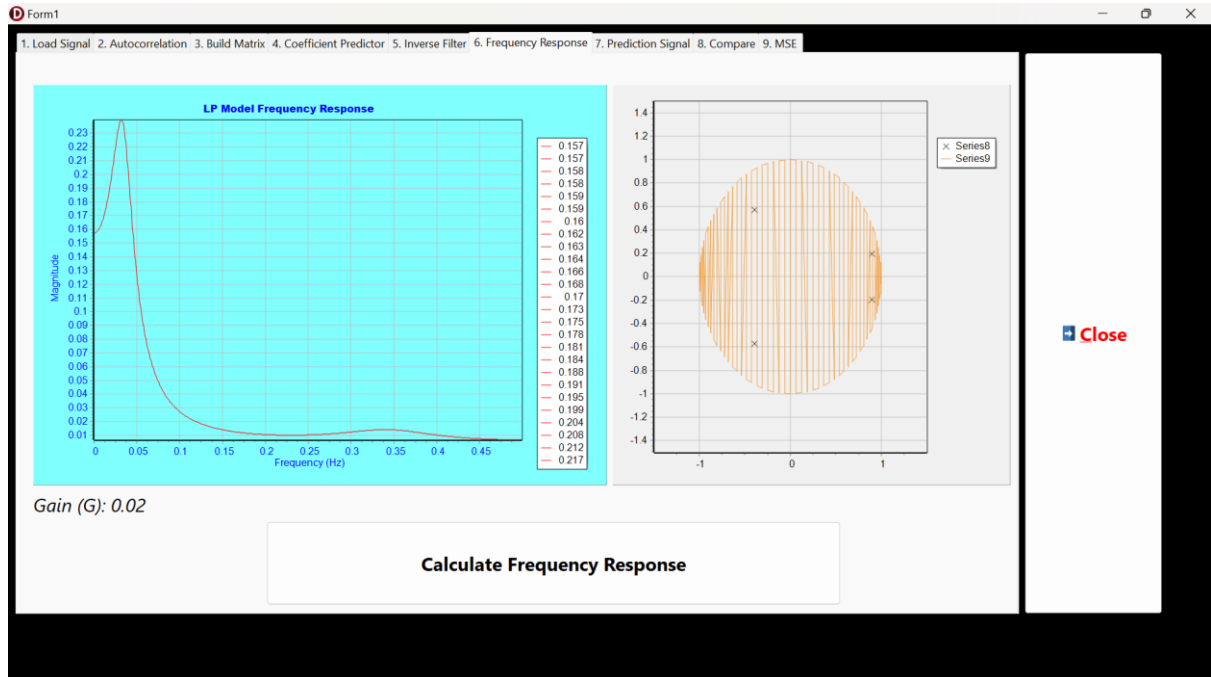


Figure 20. The Frequency Response and Pole-Zero Charts of The Linear Predictor Model without Original PCG Data and Multiplication by $10E5$, $L=4$

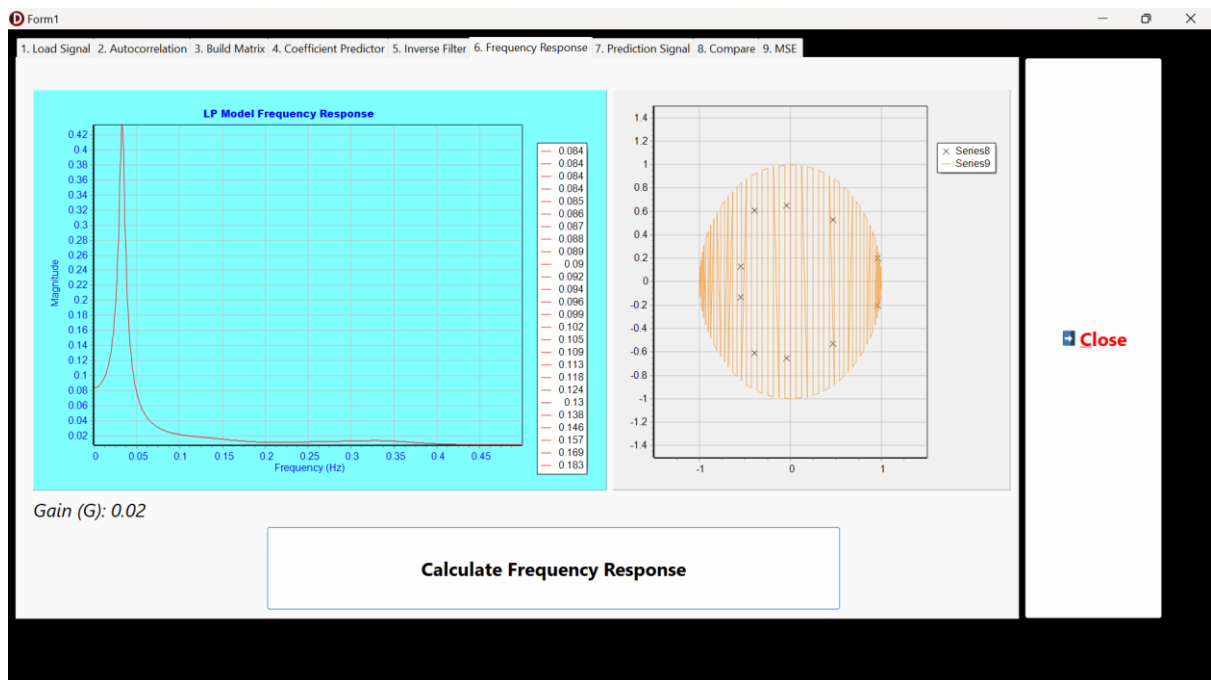


Figure 21. The Frequency Response and Pole-Zero Charts of The Linear Predictor Model without Original PCG Data and Multiplication by $10E5$, $L=10$

The **Pole-Zero Plots are identical** in both the scaled and unscaled cases. This is because the poles are derived from the $a[k]$ coefficients, which were previously shown to be scale-invariant. This proves that the poles represent the fundamental, intrinsic resonant structure of the signal, independent of its volume or amplitude.

The **Frequency Response shape is identical**, but the magnitude is scaled. The response for the unscaled signal is tiny (as shown by the Y-axis and the small Gain $G=0.02$), while the response for the scaled signal is larger by the same scaling factor. This is because the gain G is directly proportional to the signal's amplitude, while the filter's shape (defined by the poles) is not. This confirms that the scaling was a valid step that did not distort the spectral analysis.

2.3.9. Evaluation

Based on the comprehensive step-by-step results, the program is a resounding success.

1. The application correctly implements the entire workflow of a linear prediction model, from autocorrelation analysis to signal reconstruction and quantitative error evaluation.
2. The results confirm that the Linear Prediction Model is exceptionally well-suited for analysing and representing the provided PCG data. The quasi-periodic and highly correlated nature of the heart sound signal allows the model to capture its structure with a very high degree of accuracy.
3. Both the visual evidence (the perfectly overlapping original and reconstructed signals) and the quantitative metrics (an MSE value approaching zero) provide definitive proof of the model's high fidelity. The program successfully demonstrates that a complex signal can be decomposed into a set of predictive coefficients and a residual error signal, and then be perfectly reconstructed.
4. The strategy of scaling the input signal proved to be effective. It enhanced the numerical stability of the intermediate matrix calculations, making the process more robust against floating-point precision issues, without altering the fundamental, scale-invariant predictor coefficients.

While the model is highly effective, it is important to acknowledge its limitations within the context of this implementation:

1. As observed in the MSE analysis, the primary limitation is the finite precision of computer-based floating-point arithmetic. The reconstruction is not mathematically perfect (i.e., the error is not identically zero), but rather a near-perfect approximation limited by computational rounding errors. For all practical purposes in signal analysis, this error is negligible.
2. The performance of a linear prediction model is dependent on the chosen predictor order (P). In this project, $P=4$ was used for the analysis. A different order might capture the signal characteristics more or less effectively. An optimal P would model the signal's resonances without overfitting to noise. This implementation requires the user to set this value manually; a more advanced system might include algorithms (like the Akaike Information Criterion) to automatically determine the optimal order.
3. Linear prediction assumes that the statistical properties of the signal are stationary (do not change over time). While this is a reasonable assumption for a short, 2000-sample window of a PCG signal, a longer recording might contain non-stationary behavior (e.g., changes in heart rate). For such signals, the analysis would need to be done on a

frame-by-frame basis, where the coefficients are updated for each short, stationary segment.

CHAPTER III. CONCLUSION

This program involved the development of a Delphi-based computer program to form a signal of linear prediction, using a heart sound (PCG) signal as the input. The implementation successfully replicated the methodology outlined in the provided academic reference, guiding the user through eight distinct steps: signal loading, autocorrelation, matrix construction, coefficient calculation, inverse filtering, signal reconstruction, comparison, and MSE evaluation.

The results conclusively demonstrate that the linear prediction model is an exceptionally powerful and accurate tool for the analysis and synthesis of PCG data. The model was able to deconstruct the signal into a small set of predictor coefficients and a residual error signal. From these components, the original PCG waveform was reconstructed with an extremely high degree of fidelity, confirmed both visually and by a Mean Squared Error (MSE) value that was practically zero ($2.04\text{E-}025$).

For future work, this program could be enhanced by incorporating features for automatic selection of the optimal predictor order (P) and by implementing a frame-based analysis to handle longer, non-stationary signals. Furthermore, the extracted predictor coefficients and error signal characteristics could be used as features for more advanced applications, such as automated classification of heart sounds or the detection of cardiac abnormalities.

In conclusion, the project successfully achieved its objective, providing a robust tool for linear predictive analysis and a clear validation of the model's effectiveness on biomedical (PCG) signals.